```
# ==== run.py ====
# -*- coding: utf-8 -*-
"""系统启动入口。
运行：
    python run.py
然后浏览器打开 http://127.0.0.1:5000
"""
from app import create_app
from app.models import db
app = create_app()
if __name__ == "__main__":
    with app.app_context():
        db.create_all()  # 首次运行时自动建表
    app.run(host="127.0.0.1", port=5000, debug=True)
# ==== app/__init__.py ====
# -*- coding: utf-8 -*-
"""Flask 应用工厂。"""
from flask import Flask
from flask_login import LoginManager
from app.config import Config
from app.models import db, User
login_manager = LoginManager()
login_manager.login_view = "auth.login"
login_manager.login_message = "请先登录"
@login_manager.user_loader
def load_user(user_id):
    return db.session.get(User, int(user_id))
def create_app(config_class=Config):
    app = Flask(__name__)
    app.config.from_object(config_class)
    db.init_app(app)
    login_manager.init_app(app)
    # 注册视图（各模块路由）
    from app.views import auth, project, unit, standard,
    constraint, \
        ahp, indicator, evaluate, report
    app.register_blueprint(auth.bp)
    app.register_blueprint(project.bp)
    app.register_blueprint(unit.bp)
    app.register_blueprint(standard.bp)
    app.register_blueprint(constraint.bp)
    app.register_blueprint(ahp.bp)
    app.register_blueprint(indicator.bp)
    app.register_blueprint(evaluate.bp)
    app.register_blueprint(report.bp)
    # 全局登录保护：除登录页与静态资源外，均需登录
    from flask import request, redirect, url_for
    from flask_login import current_user
    PUBLIC_ENDPOINTS = {"auth.login", "static"}
    @app.before_request
```

```
    def require_login():
        if request.endpoint in PUBLIC_ENDPOINTS or
    request.endpoint is None:
            return None
        if not current_user.is_authenticated:
            return redirect(url_for("auth.login"))
        return None
    return app
# ==== app/config.py ====
# -*- coding: utf-8 -*-
"""应用配置。"""
import os
BASE_DIR = os.path.abspath(os.path.dirname(__file__))
class Config:
    SECRET_KEY = os.environ.get("SECRET_KEY",
    "mine-restoration-dev-key")
    # 单机部署，SQLite 数据库文件放在应用目录下
    SQLALCHEMY_DATABASE_URI = "sqlite:///" +
    os.path.join(BASE_DIR, "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    # 禁止跨站请求伪造（CSRF），登录与表单提交依赖此项
    WTF_CSRF_ENABLED = True
# ==== app/models.py ====
# -*- coding: utf-8 -*-
"""数据模型：系统全部数据表。
本文件由甲在阶段 0 一次性写完，作为两人独立开发的地基。
约定：两人只读此文件；确需改表时，由乙提需求、甲统一修改。
"""
import json
from datetime import datetime
from flask_login import UserMixin
from flask_sqlalchemy import SQLAlchemy
db = SQLAlchemy()
class User(UserMixin, db.Model):
    """用户（M1 用户与权限管理）。"""
    __tablename__ = "users"
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(64), unique=True,
    nullable=False, index=True)
    password_hash = db.Column(db.String(256),
    nullable=False)
    # 角色：管理员 admin / 评价员 evaluator / 查看者 viewer
    role = db.Column(db.String(20), nullable=False,
    default="viewer")
    active = db.Column(db.Boolean, nullable=False,
    default=True)
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow)
    def to_dict(self):
        return {"id": self.id, "username": self.username,
```

```
                "role": self.role, "active": self.active}
class OperationLog(db.Model):
    """操作日志（M1）。"""
    __tablename__ = "operation_logs"
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey("users.id"
    ), nullable=True)
    action = db.Column(db.String(128), nullable=False)
    # 操作类型
    target_type = db.Column(db.String(64), nullable=True)
    # 对象类型
    target_id = db.Column(db.Integer, nullable=True)
    # 对象主键
    detail = db.Column(db.Text, nullable=True)
    # 操作详情
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow, index=True)
class Project(db.Model):
    """评价项目（M2 项目与矿山档案）。"""
    __tablename__ = "projects"
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(128), nullable=False)
    code = db.Column(db.String(64), nullable=True)
    # 项目编号
    description = db.Column(db.Text, nullable=True)
    status = db.Column(db.String(20), default="进行中")
    # 进行中 / 已完成 / 已归档
    created_by = db.Column(db.Integer,
    db.ForeignKey("users.id"), nullable=True)
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow)
    patches = db.relationship("MinePatch",
    backref="project", lazy="dynamic",
                              cascade="all, delete-orphan")
class MinePatch(db.Model):
    """矿山图斑（M2）。"""
    __tablename__ = "mine_patches"
    id = db.Column(db.Integer, primary_key=True)
    project_id = db.Column(db.Integer,
    db.ForeignKey("projects.id"), nullable=False)
    name = db.Column(db.String(128), nullable=False)
    admin_region = db.Column(db.String(128), nullable=True)
    # 所在行政区
    center_x = db.Column(db.Float, nullable=True)
    # 中心经度 / 横坐标
    center_y = db.Column(db.Float, nullable=True)
    # 中心纬度 / 纵坐标
    area_ha = db.Column(db.Float, nullable=True)
    # 图斑面积（公顷）
    mineral_type = db.Column(db.String(64), nullable=True)
```

```
    # 开采矿种                                                          151
    mining_method = db.Column(db.String(64), nullable=True)           152
    # 开采方式                                                          153
    stop_years = db.Column(db.Float, nullable=True)                   154
    # 停产年限                                                          155
    responsible_entity_missing = db.Column(db.Boolean,                156
    default=True)  # 责任主体是否缺失                                   157
    created_at = db.Column(db.DateTime,                               158
    default=datetime.utcnow)                                          159
    units = db.relationship("MicroUnit", backref="patch",             160
    lazy="dynamic",                                                   161
                                cascade="all, delete-orphan")         162
class ReferenceEcosystem(db.Model):                                   163
    """参照生态系统（M2）：标准化基准值，与图斑绑定。"""                  164
    __tablename__ = "reference_ecosystems"                            165
    id = db.Column(db.Integer, primary_key=True)                      166
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat          167
    ches.id"), nullable=False)                                        168
    coverage_benchmark = db.Column(db.Float, default=75.0)            169
    # 植被覆盖度基准 %                                                170
    shannon_benchmark = db.Column(db.Float, default=0.7)              171
    # Shannon-Wiener 基准                                             172
    dominant_species = db.Column(db.String(256),                      173
    nullable=True)  # 建群种                                          174
    soil_benchmark = db.Column(db.String(256),                        175
    nullable=True)     # 土壤基准                                     176
class Standard(db.Model):                                             177
    """地方标准 / 政策文件（M3 政策与标准知识库）。"""                    178
    __tablename__ = "standards"                                       179
    id = db.Column(db.Integer, primary_key=True)                      180
    province = db.Column(db.String(32), nullable=False)               181
    # 省份                                                            182
    name = db.Column(db.String(256), nullable=False)                  183
    # 名称                                                            184
    doc_number = db.Column(db.String(128), nullable=True)             185
    # 标准编号 / 文号                                                 186
    publish_date = db.Column(db.String(32), nullable=True)            187
    implement_date = db.Column(db.String(32), nullable=True)          188
    # 类别：地方标准 / 政策文件                                       189
    category = db.Column(db.String(32), nullable=False,               190
    default="地方标准")                                               191
    summary = db.Column(db.Text, nullable=True)                       192
    # 特点概述                                                        193
    rules = db.relationship("StandardRule",                           194
    backref="standard",                                               195
                                cascade="all, delete-orphan")         196
class StandardRule(db.Model):                                         197
    """结构化规则（M3）：将标准条文拆解为可执行规则。"""                  198
    __tablename__ = "standard_rules"                                  199
    id = db.Column(db.Integer, primary_key=True)                      200
```

```
    standard_id = db.Column(db.Integer,
    db.ForeignKey("standards.id"), nullable=False)
    item = db.Column(db.String(128), nullable=False)
    # 判定项
    operator = db.Column(db.String(8), nullable=False)
    # 比较符：> >= < <= ==
    threshold = db.Column(db.String(64), nullable=True)
    # 阈值
    condition_type = db.Column(db.String(32),
    default="必要条件")  # 必要条件 / 充分条件
    region = db.Column(db.String(64), nullable=True)
    # 适用分区（宁夏分区差异化）
    note = db.Column(db.Text, nullable=True)
class ConstraintCheck(db.Model):
    """限制性判定记录（M4）。"""
    __tablename__ = "constraint_checks"
    id = db.Column(db.Integer, primary_key=True)
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat
    ches.id"), nullable=False)
    policy_pass = db.Column(db.Boolean, nullable=True)
    # 政策约束
    location_pass = db.Column(db.Boolean, nullable=True)
    # 区位约束
    pollution_pass = db.Column(db.Boolean, nullable=True)
    # 土壤污染
    geo_pass = db.Column(db.Boolean, nullable=True)
    # 地质灾害
    farmland_pass = db.Column(db.Boolean, nullable=True)
    # 耕地属性
    passed = db.Column(db.Boolean, nullable=True)
    # 总体是否通过
    reason = db.Column(db.Text, nullable=True)
    # 不通过原因
    checked_at = db.Column(db.DateTime,
    default=datetime.utcnow)
class MicroUnit(db.Model):
    """微地质单元（M5）：岩质边坡 / 碎石边坡 / 平台。"""
    __tablename__ = "micro_units"
    id = db.Column(db.Integer, primary_key=True)
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat
    ches.id"), nullable=False)
    unit_type = db.Column(db.String(32), nullable=False)
    # 单元类型
    area = db.Column(db.Float, nullable=True)
    # 面积
    ratio = db.Column(db.Float, nullable=True)
    # 面积占比（自动计算）
    description = db.Column(db.Text, nullable=True)
    # 现场描述
class Indicator(db.Model):
```

```
    """指标定义（M6）：指标树，含角色与启用单元类型。"""
    __tablename__ = "indicators"
    id = db.Column(db.Integer, primary_key=True)
    code = db.Column(db.String(32), unique=True,
    nullable=False)  # 指标编码
    name = db.Column(db.String(64), nullable=False)
    # 指标名称
    level = db.Column(db.Integer, nullable=False)
    # 1/2/3 级
    parent_code = db.Column(db.String(32), nullable=True)
    # 父指标编码
    # 角色：土壤污染指标同时作为限制性指标与评价指标
    is_constraint = db.Column(db.Boolean, default=False)
    is_evaluative = db.Column(db.Boolean, default=True)
    unit = db.Column(db.String(32), nullable=True)
    # 单位
    ideal_value = db.Column(db.String(64), nullable=True)
    # 理想值（可配置）
    ideal_type = db.Column(db.String(32), nullable=True)
    # 标准化类型：benefit/cost/mid/discrete
class WeightScheme(db.Model):
    """权重方案（M6）：按微地质单元类型分别保存。"""
    __tablename__ = "weight_schemes"
    id = db.Column(db.Integer, primary_key=True)
    unit_type = db.Column(db.String(32), nullable=False)
    # 所属单元类型
    name = db.Column(db.String(64), nullable=True)
    # 方案名称
    level = db.Column(db.Integer, nullable=False,
    default=1)        # 1=一级指标 2=二级指标
    is_default = db.Column(db.Boolean, default=False)
    # 是否默认（论文表 3）
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow)
class WeightItem(db.Model):
    """权重明细（M6）：方案内的各项权重。"""
    __tablename__ = "weight_items"
    id = db.Column(db.Integer, primary_key=True)
    scheme_id = db.Column(db.Integer,
    db.ForeignKey("weight_schemes.id"), nullable=False)
    indicator_code = db.Column(db.String(32),
    nullable=False)       # 指标编码
    weight = db.Column(db.Float, nullable=False)
    # 权重值
class ExpertMatrix(db.Model):
    """专家判断矩阵（M6）：多专家打分的原始矩阵。"""
    __tablename__ = "expert_matrices"
    id = db.Column(db.Integer, primary_key=True)
    scheme_id = db.Column(db.Integer,
    db.ForeignKey("weight_schemes.id"), nullable=False)
```

```
    expert_name = db.Column(db.String(64), nullable=True)
    # 专家姓名
    level = db.Column(db.Integer, nullable=False,
    default=1)          # 矩阵对应指标层级
    matrix_json = db.Column(db.Text, nullable=False)
    # 判断矩阵（JSON 二维数组）
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow)
class IndicatorData(db.Model):
    """指标实测数据（M7）：按微地质单元组织。"""
    __tablename__ = "indicator_data"
    id = db.Column(db.Integer, primary_key=True)
    unit_id = db.Column(db.Integer, db.ForeignKey("micro_uni
    ts.id"), nullable=False, unique=True)
    soil_ratio = db.Column(db.Float, nullable=True)
    # 土石比
    thickness = db.Column(db.Float, nullable=True)
    # 土层厚度 cm
    ph = db.Column(db.Float, nullable=True)
    organic_matter = db.Column(db.Float, nullable=True)
    # 有机质 g/kg
    # 重金属含量，JSON 字符串：{"Hg": 0.05, ...}，未检测项缺失或为 null
    heavy_metals = db.Column(db.Text, nullable=True)
    coverage = db.Column(db.Float, nullable=True)
    # 植被覆盖度 %
    shannon = db.Column(db.Float, nullable=True)
    # Shannon-Wiener 多样性指数
    slope = db.Column(db.Float, nullable=True)
    # 坡度 °
    aspect = db.Column(db.String(16), nullable=True)
    # 坡向：阳坡/阴阳坡/阴坡
    def heavy_metals_dict(self):
        return json.loads(self.heavy_metals) if
    self.heavy_metals else {}
class EvaluationResult(db.Model):
    """评价结果（M8）。"""
    __tablename__ = "evaluation_results"
    id = db.Column(db.Integer, primary_key=True)
    patch_id = db.Column(db.Integer, db.ForeignKey("mine_pat
    ches.id"), nullable=False)
    final_score = db.Column(db.Float, nullable=True)
    # 最终评价因子 F
    grade = db.Column(db.String(32), nullable=True)
    # 适宜性等级
    measures = db.Column(db.Text, nullable=True)
    # 推荐措施
    unit_scores = db.Column(db.Text, nullable=True)
    # 各单元得分 JSON
    created_at = db.Column(db.DateTime,
    default=datetime.utcnow)
```

```
class ScoreDetail(db.Model):
    """得分明细（M8）：每一层指标的权重、标准化值、加权贡献，供追溯与复核。"""
    __tablename__ = "score_details"
    id = db.Column(db.Integer, primary_key=True)
    result_id = db.Column(db.Integer,
    db.ForeignKey("evaluation_results.id"), nullable=False)
    unit_id = db.Column(db.Integer, nullable=True)
    l1_name = db.Column(db.String(64), nullable=True)
    # 一级指标
    l2_name = db.Column(db.String(64), nullable=True)
    # 二级指标
    raw_value = db.Column(db.String(64), nullable=True)
    # 实测值
    std_value = db.Column(db.Float, nullable=True)
    # 标准化值
    weight = db.Column(db.Float, nullable=True)
    # 权重
    contribution = db.Column(db.Float, nullable=True)
    # 加权贡献
# ==== app/forms.py ====
# -*- coding: utf-8 -*-
"""表单定义（共享，由甲统一维护）。
各模块的 Flask-WTF 表单集中于此。
"""
from flask_wtf import FlaskForm
from wtforms import SelectField, SubmitField, StringField,
    PasswordField, BooleanField
from wtforms.validators import DataRequired, Length
from app.utils.constants import UNIT_TYPES, ASPECT_TYPES
class AHPSelectForm(FlaskForm):
    """M6：选择微地质单元类型与指标层级，以确定判断矩阵规模。"""
    unit_type = SelectField("微地质单元类型",
    validators=[DataRequired()])
    level = SelectField("指标层级",
                        choices=[("1", "一级指标"), ("2",
    "二级指标")],
                        validators=[DataRequired()])
    l1 = SelectField("一级指标", validators=[],
    validate_choice=False)  # 动态填充，层级=2 时使用
    submit = SubmitField("生成判断矩阵")
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.unit_type.choices = [(t, t) for t in
    UNIT_TYPES]
class EmptyForm(FlaskForm):
    """仅用于承载 CSRF 令牌的空表单（矩阵录入等自定义表单使用）。"""
    pass
class LoginForm(FlaskForm):
    """M1：登录表单（占位，由乙完善）。"""
    username = StringField("用户名", validators=[DataRequired()
```

```
    , Length(max=64)])
    password = PasswordField("密码", validators=[DataRequired(
    )])
    remember = BooleanField("记住我")
    submit = SubmitField("登录")
# ==== app/utils/constants.py ====
# -*- coding: utf-8 -*-
"""评价指标常量表。
全部数值取自论文，作为系统计算的唯一基准：
- 论文 4.2.2 节：指标标准化理想极值与取值依据
- 论文 4.2.3 节 / 表 4：适宜性等级划分
- 论文表 3：按微地质单元分别确定的一级、二级指标权重
- 论文 4.2.1 节：平均随机一致性指标 RI
"""
# ------------------------------------------------------------------
    ----------------
# 一、微地质单元类型
# ------------------------------------------------------------------
    ----------------
# 论文 4.1 节："按照地形地貌划分，将其划分为平台、碎石边坡和岩质边坡
# 三种地质微单元类型"
UNIT_TYPES = ("岩质边坡", "碎石边坡", "平台")
# 每个单元类型启用的指标类别（论文表 3 隐含：岩质边坡无土壤指标，平台无地形指标）
# 供数据录入界面与评价引擎判断"该单元是否需要某类指标"
UNIT_ENABLED_INDICATORS = {
    "岩质边坡": ("植被指标", "地形指标"),          # 不设土壤指标
    "碎石边坡": ("土壤指标", "植被指标", "地形指标"),
    "平台": ("土壤指标", "植被指标"),              # 不设地形指标
}
# ------------------------------------------------------------------
    ----------------
# 二、指标标准化理想极值（论文 4.2.2 节）
# ------------------------------------------------------------------
    ----------------
IDEAL_SOIL_RATIO = 0.8          # 土石比理想值
IDEAL_THICKNESS = 20.0          # 土层厚度理想值 (cm)
IDEAL_PH_RANGE = (5.0, 9.0)     # pH 理想区间（适中型指标）
IDEAL_ORGANIC = 30.0            # 有机质理想值
    (g/kg)，全国第二次土壤普查养分等级划分标准二等最低值
IDEAL_COVERAGE = 75.0           # 植被覆盖度参照生态系统 (%)
IDEAL_SHANNON = 0.7             # Shannon-Wiener 多样性指数参照生态系统
# 重金属极值 (mg/kg)，《绿化种植土壤》CJ/T 340—2016 土壤重金属含量 IV 级最高值
# 论文原文：镉、汞、铅、铬、砷、镍、铜、锌分别为 2、2、530、400、45、220、600、800
HEAVY_METALS = ("Cd", "Hg", "Pb", "Cr", "As", "Ni", "Cu",
    "Zn")
HEAVY_METAL_LIMIT = {
    "Cd": 2.0, "Hg": 2.0, "Pb": 530.0, "Cr": 400.0,
    "As": 45.0, "Ni": 220.0, "Cu": 600.0, "Zn": 800.0,
}
# 污染指标下 8 种重金属"重要性相当，权重平均分配"（论文 4.2.1 节）
```

```
HEAVY_METAL_EQUAL_WEIGHT = 1.0 / 8.0
# 坡度分界，论文 4.2.2 节引《新一轮退耕还林还草总体方案》发改西部〔2014〕1772 号
SLOPE_BREAK = 25.0
# 坡向离散赋值，论文 4.2.2 节："阳坡、阴阳坡、阴坡分别赋值 0.2、0.6、1"
ASPECT_SCORES = {"阳坡": 0.2, "阴阳坡": 0.6, "阴坡": 1.0}
ASPECT_TYPES = ("阳坡", "阴阳坡", "阴坡")
# pH 适中型标准化的容差边界（区间外线性衰减，避免得分突变）
PH_TOL_LOW = 4.0
PH_TOL_HIGH = 10.0
# ------------------------------------------------------------------
    ------------------
# 三、适宜性等级划分（论文表 4）
# ------------------------------------------------------------------
    ------------------
GRADE_LEVELS = (
    {"grade": "不适宜", "min": None, "max": 0.3,
     "meaning": "存在限制性条件或区域生态本底较差，自然恢复影响因子严重制约自然恢复",
     "measure": "生态重建、人工辅助"},
    {"grade": "基本适宜", "min": 0.3, "max": 0.7,
     "meaning": "生态本底良好，核心自然恢复影响因子基本满足自然恢复需求",
     "measure": "自然恢复为主，可辅以封闭场地和少量补种、补植、飞播等"},
    {"grade": "适宜", "min": 0.7, "max": None,
     "meaning": "各自然恢复影响因子均能满足自然恢复需求，自然演替速度快",
     "measure": "自然恢复"},
)
def grade_of(score):
    """按论文表 4 判定适宜性等级。"""
    for g in GRADE_LEVELS:
        lo, hi = g["min"], g["max"]
        if lo is None and score < hi:
            return g
        if hi is None and score >= lo:
            return g
        if lo is not None and hi is not None and lo <=
    score < hi:
            return g
    # score 恰好等于 0.7 时落入"适宜"
    return GRADE_LEVELS[-1]
# ------------------------------------------------------------------
    ------------------
# 四、平均随机一致性指标 RI（论文 4.2.1 节给出 n=2,3,4,5，其余为常用扩展值）
# ------------------------------------------------------------------
    ------------------
RI_TABLE = {1: 0.0, 2: 0.0, 3: 0.58, 4: 0.90, 5: 1.12, 6:
    1.24,
            7: 1.32, 8: 1.41, 9: 1.45, 10: 1.49}
# 一致性比例阈值（论文：当 CR<0.1 时，判断矩阵满足一致性要求）
CR_THRESHOLD = 0.1
# ------------------------------------------------------------------
    ------------------
```

```
#  五、默认权重方案（论文表  3）
#  ----------------------------------------------------------------------
    ------------------
#  论文明确："划分岩质边坡、碎石土边坡、平台三类微地质单元开展独立评价，
#  分别计算评价指标权重"。权重由  １０  位专家打分后经  ＡＨＰ  方法确定。
#  下表为论文表  3  所载的归一化权重，作为系统默认方案。
ＤＥＦＡＵＬＴ＿ＷＥＩＧＨＴＳ  =  {
    "岩质边坡":  {
        "ｌ１":  {"植被指标":  ０．４５，  "地形指标":  ０．５５},
        "ｌ２":  {
            "植被指标":  {"植被覆盖度":  ０．６７，  "生物多样性":  ０．３３},
            "地形指标":  {"坡度":  ０．６４，  "坡向":  ０．３６},
        },
    },
    "碎石边坡":  {
        "ｌ１":  {"土壤指标":  ０．４４，  "植被指标":  ０．３１，  "地形指标":  ０．２５},
        "ｌ２":  {
            "土壤指标":  {"土石比":  ０．２５，  "土层厚度":  ０．２５，  "ｐＨ":  ０．１７，
                          "有机质":  ０．１７，  "污染指标":  ０．１７},
            "植被指标":  {"植被覆盖度":  ０．６４，  "生物多样性":  ０．３６},
            "地形指标":  {"坡度":  ０．６７，  "坡向":  ０．３３},
        },
    },
    "平台":  {
        "ｌ１":  {"土壤指标":  ０．５４，  "植被指标":  ０．４６},
        "ｌ２":  {
            "土壤指标":  {"土石比":  ０．２５，  "土层厚度":  ０．２５，  "ｐＨ":  ０．１７，
                          "有机质":  ０．１７，  "污染指标":  ０．１７},
            "植被指标":  {"植被覆盖度":  ０．６１，  "生物多样性":  ０．３９},
        },
    },
}
#  ----------------------------------------------------------------------
    ------------------
#  六、限制性指标（论文表  2  +  ４．１  节流程）
#  ----------------------------------------------------------------------
    ------------------
#  论文摘要与  ４．１  节：四项限制性指标；流程段另明确"是否为耕地"为约束条件
ＣＯＮＳＴＲＡＩＮＴ＿ＩＴＥＭＳ  =  （"政策"，  "区位约束"，  "土壤污染"，  "地质灾害"，  "耕地属性"）
#  ====  ａｐｐ/ｓｅｒｖｉｃｅｓ/ａｈｐ＿ｓｅｒｖｉｃｅ．ｐｙ  ====
#  -*-  ｃｏｄｉｎｇ:  ｕｔｆ-８  -*-
"""ＡＨＰ  权重引擎（Ｍ６）。
严格按论文  ４．２．１  节实现：
-  式（２）（３）（４）：特征根法求权重（行元素乘积  →  ｎ  次方根  →  归一化）
-  式（５）（６）（７）：一致性检验（λｍａｘ、ＣＩ、ＣＲ）
-  论文要求  ＣＲ  <  ０．１；ＲＩ  按阶数查表
"""
ｉｍｐｏｒｔ  ｎｕｍｐｙ  ａｓ  ｎｐ
ｆｒｏｍ  ａｐｐ.ｕｔｉｌｓ.ｃｏｎｓｔａｎｔｓ  ｉｍｐｏｒｔ  ＲＩ＿ＴＡＢＬＥ，  ＣＲ＿ＴＨＲＥＳＨＯＬＤ
ｄｅｆ  ｍａｔｒｉｘ＿ｆｒｏｍ＿ｕｐｐｅｒ(ｕｐｐｅｒ):
```

```
    """由判断矩阵上三角构造完整互反矩阵。
    参数
    ----
    upper : list[list[float]] 或 np.ndarray，上三角元素（不含对角线）。
        例：3 阶时传入 [[a12, a13], [a23]]。
        对角线为 1，下三角取倒数（a_ji = 1/a_ij）。
    """
    upper = np.asarray(upper, dtype=float)
    k = upper.shape[0]
    n = int(round((1 + (1 + 8 * k) ** 0.5) / 2))
    A = np.ones((n, n), dtype=float)
    idx = 0
    for i in range(n):
        for j in range(i + 1, n):
            A[i, j] = upper[idx]
            A[j, i] = 1.0 / upper[idx]
            idx += 1
    return A
def weights_and_consistency(A):
    """由判断矩阵 A 计算权重与一致性指标。
    返回
    ----
    dict：{weights: np.ndarray, lambda_max, ci, cr, n,
    consistent: bool}
    """
    A = np.asarray(A, dtype=float)
    n = A.shape[0]
    # 式(2) 行元素乘积
    row_prod = np.prod(A, axis=1)
    # 式(3) 开 n 次方根
    root = row_prod ** (1.0 / n)
    # 式(4) 归一化
    weights = root / root.sum()
    # 式(5) 最大特征值 λmax = (1/n) Σ (A·W)_i / W_i
    aw = A.dot(weights)
    lambda_max = float(np.sum(aw / weights) / n)
    # 式(6) 一致性指标 CI
    ci = (lambda_max - n) / (n - 1) if n > 1 else 0.0
    # 式(7) 一致性比例 CR = CI / RI
    ri = RI_TABLE.get(n, 1.49)
    cr = ci / ri if ri else 0.0
    return {
        "weights": weights,
        "lambda_max": lambda_max,
        "ci": ci,
        "cr": cr,
        "n": n,
        "consistent": cr < CR_THRESHOLD,
    }
def find_most_inconsistent(A):
```

```
    """定位判断矩阵中最不一致的一对比较。
    依据：完全一致矩阵应满足 a_ij = w_i / w_j。
    计算每个上三角元素与其理想值的相对偏差，返回偏差最大者。
    返回
    ----
    dict：{i, j, value, expected, deviation}，偏差最大的 (i, j)
    及对应取值。
    """
    A = np.asarray(A, dtype=float)
    r = weights_and_consistency(A)
    w = r["weights"]
    n = A.shape[0]
    best = None
    for i in range(n):
        for j in range(i + 1, n):
            expected = w[i] / w[j]
            if expected == 0:
                continue
            deviation = abs(A[i, j] - expected) / expected
            if best is None or deviation >
    best["deviation"]:
                best = {"i": i, "j": j, "value": A[i, j],
                        "expected": expected, "deviation":
    deviation}
    return best
def aggregate_matrices(matrices):
    """多专家判断矩阵聚合：几何平均（保持互反性）。
    参数
    ----
    matrices : list[np.ndarray]，多个同阶判断矩阵。
    """
    stack = np.stack([np.asarray(m, dtype=float) for m in
    matrices])
    return np.prod(stack, axis=0) ** (1.0 / len(stack))
def judge_from_weights(weights):
    """由权重向量构造完全一致判断矩阵并检验。
    用途：验证给定权重向量能否作为某个完全一致判断矩阵的主特征向量
    （A_ij = w_i / w_j 时 λmax = n、CR = 0）。
    """
    w = np.asarray(weights, dtype=float)
    n = w.shape[0]
    A = np.outer(w, 1.0 / w)
    r = weights_and_consistency(A)
    r["matrix"] = A
    return r
# ==== app/services/standardization.py ====
# -*- coding: utf-8 -*-
"""指标标准化（M8 的一部分）。
按论文 4.2.2 节式(8) 实现：
    x' = (x - x_min) / (x_max - x_min)
```

```
按指标性质分为四类：越大越优 / 越小越优 / 适中 / 离散赋值。
所有理想极值取自 app.utils.constants，可配置。
"""
from app.utils.constants import (
    IDEAL_SOIL_RATIO, IDEAL_THICKNESS, IDEAL_PH_RANGE,
    IDEAL_ORGANIC,
    IDEAL_COVERAGE, IDEAL_SHANNON, HEAVY_METAL_LIMIT,
    HEAVY_METAL_EQUAL_WEIGHT, SLOPE_BREAK, ASPECT_SCORES,
    PH_TOL_LOW, PH_TOL_HIGH,
)
def std_benefit(value, ideal):
    """越大越优指标标准化：以理想值为上限，按比例给分，饱和于 [0,1]。"""
    if value is None:
        return None
    return max(0.0, min(1.0, value / ideal))
def std_cost(value, extreme):
    """越小越优指标标准化：以极值为参考，线性折减至 [0,1]。"""
    if value is None:
        return None
    return max(0.0, min(1.0, 1.0 - value / extreme))
def std_mid_ph(value, lo, hi, tol_lo=PH_TOL_LOW,
    tol_hi=PH_TOL_HIGH):
    """适中型指标标准化（pH）：理想区间内取 1，区间外线性衰减。"""
    if value is None:
        return None
    if lo <= value <= hi:
        return 1.0
    if value > hi:
        return max(0.0, (tol_hi - value) / (tol_hi - hi))
    return max(0.0, (value - tol_lo) / (lo - tol_lo))
def std_slope(value, break_point=SLOPE_BREAK, upper=90.0):
    """坡度标准化。
    论文 4.2.2 节：以 25° 为陡坡重要分界。25° 及以下记满分 1，
    超过 25° 线性折减，90° 记 0。
    """
    if value is None:
        return None
    if value <= break_point:
        return 1.0
    return max(0.0, (upper - value) / (upper - break_point))
def aspect_score(aspect):
    """坡向离散赋值：阳坡 0.2 / 阴阳坡 0.6 / 阴坡 1.0（论文 4.2.2 节）。"""
    if aspect is None:
        return None
    return ASPECT_SCORES.get(aspect)
def pollution_score(heavy_metals):
    """污染指标：8 种重金属等权（各 0.125）合成。
    论文注：未检测 Cr、Zn 的，从矿山类型看未受污染，按无污染（=1）处理。
    heavy_metals : dict，键为金属名，值为含量 mg/kg；缺失键视为未检测。
    """
```

```
    total = 0.0                                                          701
    for metal, extreme in HEAVY_METAL_LIMIT.items():                     702
        value = heavy_metals.get(metal)                                  703
        total += std_cost(value, extreme) if value is not                704
    None else 1.0                                                        705
    return total / len(HEAVY_METAL_LIMIT)                                706
def standardize_unit(unit_type, data, weights=None):                     707
    """对单个微地质单元的全部指标做标准化。                                  708
    参数                                                                 709
    ----                                                                 710
    unit_type : str，岩质边坡 / 碎石边坡 / 平台                            711
    data : dict，实测数据，字段见接口契约：                                 712
        soil_ratio, thickness, ph, organic_matter,                       713
    heavy_metals(dict),                                                  714
        coverage, shannon, slope, aspect                                 715
    weights : dict，该单元类型的权重（app.utils.constants.DEFAULT_WEIG    716
    HTS 结构），                                                          717
            未提供时用默认值。                                            718
    返回                                                                 719
    ----                                                                 720
    dict：{ 一级指标： { 二级指标： （实测值， 标准化值， 权重， 贡献） } }   721
    标准化值可能为 None，表示该指标不适用或方法待定。                       722
    """                                                                  723
    from app.utils.constants import DEFAULT_WEIGHTS                      724
    if weights is None:                                                  725
        weights = DEFAULT_WEIGHTS[unit_type]                             726
    l1_enabled = weights["l1"].keys()                                    727
    l2w = weights["l2"]                                                  728
    result = {}                                                          729
    # ---- 土壤指标 ----                                                730
    if "土壤指标" in l1_enabled:                                          731
        result["土壤指标"] = {                                            732
            "土石比": (data.get("soil_ratio"),                           733
                       std_benefit(data.get("soil_ratio"),               734
    IDEAL_SOIL_RATIO),                                                   735
                       l2w["土壤指标"]["土石比"]),                        736
            "土层厚度": (data.get("thickness"),                          737
                         std_benefit(data.get("thickness"),              738
    IDEAL_THICKNESS),                                                    739
                         l2w["土壤指标"]["土层厚度"]),                    740
            "pH": (data.get("ph"),                                       741
                   std_mid_ph(data.get("ph"),                            742
    *IDEAL_PH_RANGE),                                                    743
                   l2w["土壤指标"]["pH"]),                               744
            "有机质": (data.get("organic_matter"),                       745
                       std_benefit(data.get("organic_matter")            746
    , IDEAL_ORGANIC),                                                    747
                       l2w["土壤指标"]["有机质"]),                       748
            "污染指标": (data.get("heavy_metals") or {},                 749
                         pollution_score(data.get("heavy_meta            750
```

```
        ls") or {}),                                                    751
                                    l2w["土壤指标"]["污染指标"]),          752
            }                                                           753
        # ---- 植被指标 ----                                           754
        if "植被指标" in l1_enabled:                                    755
            result["植被指标"] = {                                      756
                "植被覆盖度": (data.get("coverage"),                    757
                                    std_benefit(data.get("coverage"),   758
        IDEAL_COVERAGE),                                                759
                                    l2w["植被指标"]["植被覆盖度"]),      760
                "生物多样性": (data.get("shannon"),                     761
                                    std_benefit(data.get("shannon"),    762
        IDEAL_SHANNON),                                                 763
                                    l2w["植被指标"]["生物多样性"]),      764
            }                                                           765
        # ---- 地形指标 ----                                           766
        if "地形指标" in l1_enabled:                                    767
            result["地形指标"] = {                                      768
                "坡度": (data.get("slope"),                             769
                        std_slope(data.get("slope")),                   770
                        l2w["地形指标"]["坡度"]),                       771
                "坡向": (data.get("aspect"),                            772
                        aspect_score(data.get("aspect")),               773
                        l2w["地形指标"]["坡向"]),                       774
            }                                                           775
        return result                                                   776
# ==== app/services/evaluation_service.py ====                          777
# -*- coding: utf-8 -*-                                                 778
"""适宜性评价引擎（M8）。                                               779
按论文式(9) 实现：                                                      780
- 单元级：F' = Σ（二级指标权重 × 该二级指标下三级指标标准化值）          781
- 矿山级：F = Σ(Si × F'i)，Si 为各微地质单元面积占比                    782
- 等级判定按论文表 4。                                                  783
"""                                                                     784
from app.utils.constants import DEFAULT_WEIGHTS, grade_of               785
from app.services import standardization as std                         786
def score_unit(unit_type, data, weights=None):                          787
    """计算单个微地质单元的评价因子 F'。                                788
    返回                                                                789
    ----                                                                790
    dict：{ l1_scores: {一级指标: 得分}, unit_score: F', details:       791
    [...] }                                                             792
    若某一级指标下存在"不适用"（标准化值为 None）的二级指标，           793
    则按其余项权重归一化后计算，保证单元得分始终有定义。                794
    """                                                                 795
    if weights is None:                                                 796
        weights = DEFAULT_WEIGHTS[unit_type]                            797
    std_result = std.standardize_unit(unit_type, data,                  798
    weights)                                                            799
    l1_weights = weights["l1"]                                          800
```

```
    l1_scores = {}
    details = []
    for l1_name, items in std_result.items():
        wsum = 0.0
        acc = 0.0
        for l2_name, (raw, std_value, w2) in items.items():
            if std_value is None:
                # 不适用项：不计入
                details.append({
                    "l1": l1_name, "l2": l2_name, "raw":
raw,
                    "std_value": None, "weight": w2,
"contribution": None,
                })
                continue
            contribution = std_value * w2
            acc += contribution
            wsum += w2
            details.append({
                "l1": l1_name, "l2": l2_name, "raw": raw,
                "std_value": std_value, "weight": w2,
                "contribution": contribution,
            })
        l1_scores[l1_name] = acc / wsum if wsum > 0 else 0.0
    unit_score = sum(l1_weights[k] * l1_scores[k] for k in
l1_scores)
    return {"l1_scores": l1_scores, "unit_score":
unit_score, "details": details}
def score_mine(units):
    """计算整个矿山的评价因子 F 与适宜性等级。
    参数
    ----
    units : list[dict]，每个元素：
        {unit_type, ratio, data}
        ratio 为该单元面积占比（0~1），data 为实测数据 dict。
    返回
    ----
    dict:{ final_score, grade, grade_info, unit_results }
    """
    total = 0.0
    unit_results = []
    for u in units:
        r = score_unit(u["unit_type"], u["data"])
        weighted = u["ratio"] * r["unit_score"]
        total += weighted
        unit_results.append({
            "unit_type": u["unit_type"],
            "ratio": u["ratio"],
            "unit_score": r["unit_score"],
            "weighted": weighted,
```

```
                "l1_scores": r["l1_scores"],
                "details": r["details"],
            })
    grade_info = grade_of(total)
    return {
        "final_score": total,
        "grade": grade_info["grade"],
        "grade_info": grade_info,
        "unit_results": unit_results,
    }
# ==== app/services/report_service.py ====
# -*- coding: utf-8 -*-
"""报告生成（M9）：生成 Word 评价报告。
依赖 python-docx。报告结构：
标题 → 图斑信息 → 评价结论 → 各微地质单元得分 → 完整得分明细 → 生成时间。
"""
import io
from datetime import datetime
from docx import Document
from docx.shared import Pt, Cm
from docx.oxml.ns import qn
from docx.enum.text import WD_ALIGN_PARAGRAPH
def _set_run(run, text=None, size=12, bold=False):
    if text is not None:
        run.text = text
    run.font.name = "Times New Roman"
    run.font.size = Pt(size)
    run.font.bold = bold
    # 中文字体
    run._element.rPr.rFonts.set(qn("w:eastAsia"), "宋体")
    return run
def _para(doc, text, size=12, bold=False, align=None,
    space_after=6):
    p = doc.add_paragraph()
    if align is not None:
        p.alignment = align
    p.paragraph_format.space_after = Pt(space_after)
    _set_run(p.add_run(), text, size, bold)
    return p
def _heading(doc, text, level=1):
    h = doc.add_heading(text, level=level)
    for run in h.runs:
        run.font.name = "Times New Roman"
        run._element.rPr.rFonts.set(qn("w:eastAsia"), "黑体")
    return h
def _table(doc, headers, rows, widths=None):
    t = doc.add_table(rows=1, cols=len(headers))
    t.style = "Table Grid"
    hdr = t.rows[0].cells
    for i, htext in enumerate(headers):
```

```
            _set_run(hdr[i].paragraphs[0].add_run(), htext,          901
        10.5, bold=True)                                             902
        for row in rows:                                             903
            cells = t.add_row().cells                                904
            for i, val in enumerate(row):                            905
                _set_run(cells[i].paragraphs[0].add_run(),           906
        str(val), 10.5)                                              907
        return t                                                     908
def build_report(patch, mine_result):                                909
        """生成评价报告 Word 文档，返回 BytesIO。                    910
        参数                                                         911
        ----                                                         912
        patch : MinePatch，图斑对象                                  913
        mine_result : evaluation_service.score_mine 的返回 dict      914
        """                                                          915
        doc = Document()                                             916
        # 页边距                                                     917
        for section in doc.sections:                                 918
            section.top_margin = Cm(2.5)                             919
            section.bottom_margin = Cm(2.5)                          920
            section.left_margin = Cm(2.5)                            921
            section.right_margin = Cm(2.5)                           922
        _heading(doc, "历史遗留废弃矿山自然恢复适宜性评价报告", 0)   923
        _para(doc, "软件：历史遗留废弃矿山自然恢复适宜性评价系统 Ｖ1.0",  924
                size=10.5, align=WD_ALIGN_PARAGRAPH.CENTER)          925
        _para(doc, "生成时间：%s" % datetime.now().strftime("%Y-%m-%d  926
        %H:%M:%S"),                                                  927
                size=10.5, align=WD_ALIGN_PARAGRAPH.CENTER,          928
        space_after=12)                                              929
        # 图斑信息                                                   930
        _heading(doc, "一、图斑基本信息", 1)                         931
        info = [                                                     932
            ("图斑名称", patch.name),                                933
            ("所在行政区", patch.admin_region or "—"),               934
            ("图斑面积（公顷）", str(patch.area_ha) if patch.area_ha is  935
        not None else "—"),                                          936
            ("开采矿种", patch.mineral_type or "—"),                 937
            ("开采方式", patch.mining_method or "—"),                938
        ]                                                            939
        _table(doc, ["项目", "内容"], info)                          940
        # 评价结论                                                   941
        _heading(doc, "二、评价结论", 1)                             942
        gi = mine_result["grade_info"]                               943
        concl = [                                                    944
            ("最终评价因子 F", "%.4f" % mine_result["final_score"]),  945
            ("适宜性等级", mine_result["grade"]),                    946
            ("判定依据", gi["meaning"]),                             947
            ("推荐措施", gi["measure"]),                             948
        ]                                                            949
        _table(doc, ["项目", "结果"], concl)                         950
```

```
    # 各单元得分
    _heading(doc, "三、各微地质单元得分", 1)
    rows = []
    for u in mine_result["unit_results"]:
        rows.append((u["unit_type"], "%.2f%%" % (u["ratio"] * 100),
                     "%.4f" % u["unit_score"], "%.4f" % u["weighted"]))
    _table(doc, ["微地质单元", "面积占比", "单元评价因子 F'", "加权贡献"], rows)
    # 完整得分明细
    _heading(doc, "四、得分明细", 1)
    for u in mine_result["unit_results"]:
        _para(doc, "%s（占比 %.2f%%）" % (u["unit_type"], u["ratio"] * 100),
              size=11, bold=True, space_after=4)
        detail_rows = []
        for d in u["details"]:
            raw = "—" if d["raw"] is None else d["raw"]
            std = "—" if d["std_value"] is None else "%.4f" % d["std_value"]
            w = "%.4f" % d["weight"]
            c = "—" if d["contribution"] is None else "%.4f" % d["contribution"]
            detail_rows.append((d["l1"], d["l2"], raw, std, w, c))
        _table(doc, ["一级指标", "二级指标", "实测值", "标准化值", "权重", "贡献"],
               detail_rows)
        # 一级指标得分
        l1_row = []
        for k, v in u["l1_scores"].items():
            l1_row.append("%s=%.4f" % (k, v))
        _para(doc, "一级指标得分：" + "，".join(l1_row), size=10.5, space_after=10)
    buf = io.BytesIO()
    doc.save(buf)
    buf.seek(0)
    return buf
# ==== app/views/auth.py ====
# -*- coding: utf-8 -*-
"""M1 用户与权限管理 —— 登录、注销、权限控制。"""
from flask import Blueprint, render_template, redirect,
    url_for, request, flash
from flask_login import login_user, logout_user,
    login_required, current_user
from werkzeug.security import check_password_hash,
    generate_password_hash
from app.forms import LoginForm
from app.models import db, User
```

```
bp = Blueprint("auth", __name__)
@bp.route("/")
def index():
    if current_user.is_authenticated:
        return render_template("auth/home.html")
    return redirect(url_for("auth.login"))
@bp.route("/login", methods=["GET", "POST"])
def login():
    if current_user.is_authenticated:
        return redirect(url_for("auth.index"))
    form = LoginForm()
    if form.validate_on_submit():
        user = User.query.filter_by(username=form.username.data).first()
        if user and check_password_hash(user.password_hash, form.password.data):
            login_user(user, remember=form.remember.data)
            flash("登录成功", "ok")
            return redirect(url_for("auth.index"))
        flash("用户名或密码错误", "error")
    return render_template("auth/login.html", form=form)
@bp.route("/logout")
@login_required
def logout():
    logout_user()
    flash("已退出登录", "ok")
    return redirect(url_for("auth.login"))
@bp.route("/users")
@login_required
def users():
    """用户列表（管理员可管理）。"""
    all_users = User.query.order_by(User.id).all()
    return render_template("auth/users.html",
    users=all_users)
@bp.route("/users/add", methods=["GET", "POST"])
@login_required
def add_user():
    if current_user.role != "admin":
        flash("仅管理员可新增用户", "error")
        return redirect(url_for("auth.users"))
    if request.method == "POST":
        username = request.form.get("username", "").strip()
        password = request.form.get("password", "")
        role = request.form.get("role", "viewer")
        if not username or not password:
            flash("用户名和密码不能为空", "error")
            return redirect(url_for("auth.add_user"))
        if User.query.filter_by(username=username).first():
            flash("用户名已存在", "error")
            return redirect(url_for("auth.add_user"))
```

```
            user = User(username=username,                                  1051
                        password_hash=generate_password_hash(pas            1052
        sword), role=role)                                                  1053
            db.session.add(user)                                            1054
            db.session.commit()                                             1055
            flash("用户已添加", "ok")                                        1056
            return redirect(url_for("auth.users"))                          1057
        return render_template("auth/add_user.html")                        1058
# ==== app/views/project.py ====                                            1059
# -*- coding: utf-8 -*-                                                     1060
"""M2 项目与矿山档案 —— 项目、图斑、参照生态系统。"""                          1061
from flask import Blueprint, render_template, request,                      1062
    redirect, url_for, flash                                                1063
from flask_login import login_required, current_user                        1064
from app.forms import EmptyForm                                             1065
from app.models import db, Project, MinePatch,                              1066
    ReferenceEcosystem                                                      1067
bp = Blueprint("project", __name__, url_prefix="/project")                  1068
@bp.route("/")                                                              1069
@login_required                                                             1070
def index():                                                                1071
    projects = Project.query.order_by(Project.id.desc()).all                1072
    ()                                                                      1073
    return render_template("project/index.html",                            1074
    projects=projects)                                                      1075
@bp.route("/add", methods=["GET", "POST"])                                  1076
@login_required                                                             1077
def add():                                                                  1078
    form = EmptyForm()                                                      1079
    if request.method == "POST" and form.validate_on_submit(                1080
    ):                                                                      1081
        p = Project(                                                        1082
            name=request.form.get("name", "").strip(),                      1083
            code=request.form.get("code", "").strip() or                    1084
    None,                                                                   1085
            description=request.form.get("description",                     1086
    "").strip() or None,                                                    1087
            created_by=current_user.id,                                     1088
        )                                                                   1089
        if not p.name:                                                      1090
            flash("项目名称不能为空", "error")                               1091
        else:                                                               1092
            db.session.add(p)                                               1093
            db.session.commit()                                             1094
            flash("项目已创建", "ok")                                        1095
            return redirect(url_for("project.index"))                       1096
    return render_template("project/edit.html", form=form,                  1097
    project=None)                                                           1098
@bp.route("/<int:pid>/edit", methods=["GET", "POST"])                       1099
@login_required                                                             1100
```

```
def edit(pid):                                                    1101
    p = Project.query.get_or_404(pid)                             1102
    form = EmptyForm()                                            1103
    if request.method == "POST" and form.validate_on_submit(      1104
    ):                                                            1105
        p.name = request.form.get("name", "").strip()             1106
        p.code = request.form.get("code", "").strip() or          1107
    None                                                          1108
        p.description = request.form.get("description",           1109
    "").strip() or None                                           1110
        p.status = request.form.get("status", "进行中")           1111
        db.session.commit()                                       1112
        flash("项目已保存", "ok")                                 1113
        return redirect(url_for("project.index"))                 1114
    return render_template("project/edit.html", form=form,        1115
    project=p)                                                    1116
@bp.route("/<int:pid>/delete", methods=["POST"])                  1117
@login_required                                                   1118
def delete(pid):                                                  1119
    p = Project.query.get_or_404(pid)                             1120
    db.session.delete(p)                                          1121
    db.session.commit()                                           1122
    flash("项目已删除", "ok")                                     1123
    return redirect(url_for("project.index"))                     1124
# ------------------------------ 矿山图斑 ----------------------------1125
    ----                                                          1126
@bp.route("/<int:pid>/patches")                                   1127
@login_required                                                   1128
def patches(pid):                                                 1129
    p = Project.query.get_or_404(pid)                             1130
    return render_template("project/patches.html",                1131
    project=p)                                                    1132
@bp.route("/<int:pid>/patch/add", methods=["GET", "POST"])        1133
@login_required                                                   1134
def patch_add(pid):                                               1135
    p = Project.query.get_or_404(pid)                             1136
    form = EmptyForm()                                            1137
    if request.method == "POST" and form.validate_on_submit(      1138
    ):                                                            1139
        def fv(name, cast=str):                                   1140
            v = request.form.get(name, "").strip()                1141
            return cast(v) if v != "" else None                   1142
        patch = MinePatch(                                        1143
            project_id=p.id,                                      1144
            name=request.form.get("name", "").strip(),            1145
            admin_region=fv("admin_region"),                      1146
            area_ha=fv("area_ha", float),                         1147
            mineral_type=fv("mineral_type"),                      1148
            mining_method=fv("mining_method"),                    1149
            stop_years=fv("stop_years", float),                   1150
```

```
            responsible_entity_missing=bool(request.form.get
    ("responsible_entity_missing")),
        )
        if not patch.name:
            flash("图斑名称不能为空", "error")
        else:
            db.session.add(patch)
            db.session.flush()
            #  参照生态系统默认取论文值
            db.session.add(ReferenceEcosystem(
                patch_id=patch.id, coverage_benchmark=75.0,
    shannon_benchmark=0.7))
            db.session.commit()
            flash("图斑已创建", "ok")
            return redirect(url_for("project.patches",
    pid=p.id))
    return render_template("project/patch_edit.html",
    form=form, project=p, patch=None)
@bp.route("/patch/<int:patch_id>/edit", methods=["GET",
    "POST"])
@login_required
def patch_edit(patch_id):
    patch = MinePatch.query.get_or_404(patch_id)
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        def fv(name, cast=str):
            v = request.form.get(name, "").strip()
            return cast(v) if v != "" else None
        patch.name = request.form.get("name", "").strip()
        patch.admin_region = fv("admin_region")
        patch.area_ha = fv("area_ha", float)
        patch.mineral_type = fv("mineral_type")
        patch.mining_method = fv("mining_method")
        patch.stop_years = fv("stop_years", float)
        patch.responsible_entity_missing =
    bool(request.form.get("responsible_entity_missing"))
        #  参照生态系统
        ref = ReferenceEcosystem.query.filter_by(patch_id=pa
    tch.id).first()
        if ref is None:
            ref = ReferenceEcosystem(patch_id=patch.id)
            db.session.add(ref)
        ref.coverage_benchmark = fv("coverage_benchmark",
    float) or 75.0
        ref.shannon_benchmark = fv("shannon_benchmark",
    float) or 0.7
        ref.dominant_species = fv("dominant_species")
        db.session.commit()
        flash("图斑已保存", "ok")
```

```
            return redirect(url_for("project.patches",                1201
        pid=patch.project_id))                                        1202
        return render_template("project/patch_edit.html",             1203
        form=form,                                                    1204
                                    project=patch.project,            1205
        patch=patch)                                                  1206
# ==== app/views/unit.py ====                                         1207
# -*- coding: utf-8 -*-                                               1208
"""M5 微地质单元管理 —— 岩质边坡 / 碎石边坡 / 平台的划分与面积占比。"""          1209
from flask import Blueprint, render_template, request,                1210
    redirect, url_for, flash                                          1211
from flask_login import login_required                                1212
from app.forms import EmptyForm                                       1213
from app.models import db, MinePatch, MicroUnit                       1214
from app.utils.constants import UNIT_TYPES                            1215
bp = Blueprint("unit", __name__, url_prefix="/unit")                  1216
def recalc_ratio(patch_id):                                           1217
    """按面积重算该图斑各单元面积占比。"""                                    1218
    units = MicroUnit.query.filter_by(patch_id=patch_id).all          1219
    ()                                                                1220
    total = sum(u.area or 0.0 for u in units)                         1221
    if total > 0:                                                     1222
        for u in units:                                               1223
            u.ratio = round((u.area or 0.0) / total, 6)               1224
    return total                                                      1225
@bp.route("/<int:patch_id>")                                          1226
@login_required                                                       1227
def index(patch_id):                                                  1228
    patch = MinePatch.query.get_or_404(patch_id)                      1229
    units = MicroUnit.query.filter_by(patch_id=patch_id).all          1230
    ()                                                                1231
    total = sum(u.area or 0.0 for u in units)                         1232
    ratio_sum = sum(u.ratio or 0.0 for u in units)                    1233
    return render_template("unit/index.html", patch=patch,            1234
    units=units,                                                      1235
                                    total_area=total,                 1236
    ratio_sum=ratio_sum,                                              1237
                                    unit_types=UNIT_TYPES,            1238
    form=EmptyForm())                                                 1239
@bp.route("/<int:patch_id>/add", methods=["POST"])                    1240
@login_required                                                       1241
def add(patch_id):                                                    1242
    patch = MinePatch.query.get_or_404(patch_id)                      1243
    form = EmptyForm()                                                1244
    if form.validate_on_submit():                                     1245
        unit_type = request.form.get("unit_type")                     1246
        area = request.form.get("area", "").strip()                   1247
        u = MicroUnit(patch_id=patch.id,                              1248
    unit_type=unit_type,                                              1249
                          area=float(area) if area else None,         1250
```

```
                            description=request.form.get("descript
    ion", "").strip() or None)
            db.session.add(u)
            db.session.flush()
            recalc_ratio(patch.id)
            db.session.commit()
            flash("微地质单元已添加", "ok")
    return redirect(url_for("unit.index",
    patch_id=patch_id))
@bp.route("/edit/<int:unit_id>", methods=["POST"])
@login_required
def edit(unit_id):
    u = MicroUnit.query.get_or_404(unit_id)
    form = EmptyForm()
    if form.validate_on_submit():
        u.unit_type = request.form.get("unit_type",
    u.unit_type)
        area = request.form.get("area", "").strip()
        u.area = float(area) if area else None
        u.description = request.form.get("description",
    "").strip() or None
        recalc_ratio(u.patch_id)
        db.session.commit()
        flash("已保存", "ok")
    return redirect(url_for("unit.index",
    patch_id=u.patch_id))
@bp.route("/delete/<int:unit_id>", methods=["POST"])
@login_required
def delete(unit_id):
    u = MicroUnit.query.get_or_404(unit_id)
    pid = u.patch_id
    db.session.delete(u)
    db.session.flush()
    recalc_ratio(pid)
    db.session.commit()
    flash("已删除", "ok")
    return redirect(url_for("unit.index", patch_id=pid))
# ==== app/views/standard.py ====
# -*- coding: utf-8 -*-
"""M3 政策与标准知识库 —— 标准/政策文件的结构化规则与省际口径比对。"""
from flask import Blueprint, render_template, request,
    redirect, url_for, flash
from flask_login import login_required
from app.forms import EmptyForm
from app.models import db, Standard, StandardRule
bp = Blueprint("standard", __name__, url_prefix="/standard")
@bp.route("/")
@login_required
def index():
    standards = Standard.query.order_by(Standard.category,
```

```
    Standard.province).all()
    local = [s for s in standards if s.category == "地方标准"]
    policy = [s for s in standards if s.category != "地方标准"]
    return render_template("standard/index.html",
    local=local, policy=policy)
@bp.route("/add", methods=["GET", "POST"])
@login_required
def add():
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        s = Standard(
            province=request.form.get("province",
    "").strip(),
            name=request.form.get("name", "").strip(),
            doc_number=request.form.get("doc_number",
    "").strip() or None,
            publish_date=request.form.get("publish_date",
    "").strip() or None,
            implement_date=request.form.get("implement_date"
    , "").strip() or None,
            category=request.form.get("category", "地方标准"),
            summary=request.form.get("summary", "").strip()
    or None,
        )
        if not s.province or not s.name:
            flash("省份与名称不能为空", "error")
        else:
            db.session.add(s)
            db.session.commit()
            flash("标准已添加", "ok")
            return redirect(url_for("standard.index"))
    return render_template("standard/edit.html", form=form,
    standard=None)
@bp.route("/<int:sid>/edit", methods=["GET", "POST"])
@login_required
def edit(sid):
    s = Standard.query.get_or_404(sid)
    form = EmptyForm()
    if request.method == "POST" and form.validate_on_submit(
    ):
        s.province = request.form.get("province",
    "").strip()
        s.name = request.form.get("name", "").strip()
        s.doc_number = request.form.get("doc_number",
    "").strip() or None
        s.publish_date = request.form.get("publish_date",
    "").strip() or None
        s.implement_date = request.form.get("implement_date"
    , "").strip() or None
```

```
            s.category = request.form.get("category", "地方标准")
            s.summary = request.form.get("summary", "").strip()
        or None
            db.session.commit()
            flash("已保存", "ok")
            return redirect(url_for("standard.rules", sid=s.id))
        return render_template("standard/edit.html", form=form,
        standard=s)
@bp.route("/<int:sid>/delete", methods=["POST"])
@login_required
def delete(sid):
        s = Standard.query.get_or_404(sid)
        StandardRule.query.filter_by(standard_id=sid).delete()
        db.session.delete(s)
        db.session.commit()
        flash("已删除", "ok")
        return redirect(url_for("standard.index"))
@bp.route("/<int:sid>/rules", methods=["GET", "POST"])
@login_required
def rules(sid):
        s = Standard.query.get_or_404(sid)
        form = EmptyForm()
        if request.method == "POST" and form.validate_on_submit(
        ):
            r = StandardRule(
                standard_id=s.id,
                item=request.form.get("item", "").strip(),
                operator=request.form.get("operator", ">="),
                threshold=request.form.get("threshold",
        "").strip() or None,
                condition_type=request.form.get("condition_type"
        , "必要条件"),
                region=request.form.get("region", "").strip()
        or None,
                note=request.form.get("note", "").strip() or
        None,
            )
            if not r.item:
                flash("判定项不能为空", "error")
            else:
                db.session.add(r)
                db.session.commit()
                flash("规则已添加", "ok")
            return redirect(url_for("standard.rules", sid=s.id))
        rs = StandardRule.query.filter_by(standard_id=s.id).all(
        )
        return render_template("standard/rules.html",
        form=form, standard=s, rules=rs)
@bp.route("/rule/<int:rid>/delete", methods=["POST"])
@login_required
```

```
def rule_delete(rid):
    r = StandardRule.query.get_or_404(rid)
    sid = r.standard_id
    db.session.delete(r)
    db.session.commit()
    flash("规则已删除", "ok")
    return redirect(url_for("standard.rules", sid=sid))
@bp.route("/compare")
@login_required
def compare():
    """省际口径比对：同一指标在不同省份的阈值横向对比。"""
    rules = StandardRule.query.join(Standard).all()
    groups = {}
    for r in rules:
        groups.setdefault(r.item, []).append(r)
    return render_template("standard/compare.html",
    groups=groups)
# ==== app/views/constraint.py ====
# -*- coding: utf-8 -*-
"""M4 限制性指标判定 —— 一票否决判定。
按论文表 2 与 4.1 节流程，共 5 项：政策、区位约束、土壤污染、地质灾害、耕地属性。
任一项不通过即判定为"人工辅助或生态重建"，不再进入定量评价。
"""
from flask import Blueprint, render_template, request,
    redirect, url_for, flash
from flask_login import login_required
from app.forms import EmptyForm
from app.models import db, MinePatch, ConstraintCheck
from app.utils.constants import CONSTRAINT_ITEMS
bp = Blueprint("constraint", __name__,
    url_prefix="/constraint")
# 字段名 -> 中文名
FIELDS = [
    ("policy_pass", "政策约束"),
    ("location_pass", "区位约束"),
    ("pollution_pass", "土壤污染"),
    ("geo_pass", "地质灾害"),
    ("farmland_pass", "耕地属性"),
]
@bp.route("/")
@login_required
def index():
    patches = MinePatch.query.order_by(MinePatch.id.desc()).
    all()
    latest = {}
    for p in patches:
        latest[p.id] = ConstraintCheck.query.filter_by(patch
    _id=p.id) \
            .order_by(ConstraintCheck.id.desc()).first()
    return render_template("constraint/index.html",
```

```
    patches=patches, latest=latest)
@bp.route("/<int:patch_id>", methods=["GET", "POST"])
@login_required
def check(patch_id):
    patch = MinePatch.query.get_or_404(patch_id)
    form = EmptyForm()
    record = ConstraintCheck.query.filter_by(patch_id=patch_
    id) \
        .order_by(ConstraintCheck.id.desc()).first()
    if request.method == "POST" and form.validate_on_submit(
    ):
        if record is None:
            record = ConstraintCheck(patch_id=patch.id)
            db.session.add(record)
        for field, _ in FIELDS:
            setattr(record, field, request.form.get(field)
    == "1")
        record.passed = all(getattr(record, f) for f, _ in
    FIELDS)
        failed = [name for f, name in FIELDS if not
    getattr(record, f)]
        record.reason = ("全部通过" if record.passed
                         else "未通过：" + "、".join(failed))
        db.session.commit()
        flash("判定结果已保存", "ok")
        return redirect(url_for("constraint.check",
    patch_id=patch.id))
    # 预处理各条件的当前取值，供模板直接使用（Jinja 无 getattr）
    field_states = [(f, name, None if record is None else
    getattr(record, f))
                    for f, name in FIELDS]
    return render_template("constraint/check.html",
    patch=patch, form=form,
                           fields=FIELDS,
    field_states=field_states,
                           record=record)
# ==== app/views/ahp.py ====
# -*- coding: utf-8 -*-
"""M6 指标体系与 AHP 权重 —— 路由与页面逻辑。"""
from flask import Blueprint, render_template, request,
    redirect, url_for, flash
from app.forms import AHPSelectForm, EmptyForm
from app.utils.constants import DEFAULT_WEIGHTS
from app.services import ahp_service as ahp
bp = Blueprint("ahp", __name__, url_prefix="/ahp")
def indicator_list(unit_type, level, l1=None):
    """返回 （指标名称列表， 默认权重列表）。
    level=1 时返回一级指标；level=2 时返回指定一级指标下的二级指标。
    """
    weights = DEFAULT_WEIGHTS[unit_type]
```

```
      <tr>                                                                    2382
        <td>{{ s.province }}</td>                                             2383
        <td style="text-align:left;">{{ s.name }}</td>                        2384
        <td class="mono">{{ s.doc_number or "—" }}</td>                       2385
        <td>{{ s.rules|length }}</td>                                         2386
        <td><a href="/standard/{{ s.id }}/rules">规则</a> | <a                2387
      href="/standard/{{ s.id }}/edit">编辑</a></td>                          2388
      </tr>                                                                   2389
      {% else %}                                                              2390
      <tr><td colspan="5" class="muted">暂无</td></tr>                        2391
      {% endfor %}                                                            2392
    </table>                                                                  2393
</div>                                                                        2394
{% endblock %}                                                                2395
# ==== app/templates/standard/edit.html ====                                  2396
{% extends "base.html" %}                                                     2397
{% block title %}{{ "编辑标准" if standard else "添加标准" }}{%               2398
      endblock %}                                                             2399
{% block content %}                                                           2400
<div class="card">                                                            2401
    <h2>{{ "编辑标准" if standard else "添加标准" }}</h2>                     2402
    <form method="post">                                                      2403
      {{ form.hidden_tag() }}                                                 2404
      <table style="width:auto;">                                             2405
        <tr><td style="text-align:left;">省份</td>                            2406
            <td><input type="text" name="province" value="{{                  2407
      standard.province if standard else '' }}"                               2408
      required></td></tr>                                                     2409
        <tr><td style="text-align:left;">名称</td>                            2410
            <td><input type="text" name="name"                                2411
      style="width:420px;" value="{{ standard.name if                         2412
      standard else '' }}" required></td></tr>                                2413
        <tr><td style="text-align:left;">标准编号/文号</td>                   2414
            <td><input type="text" name="doc_number"                          2415
      style="width:300px;" value="{{ standard.doc_number or                   2416
      '' if standard else '' }}"></td></tr>                                   2417
        <tr><td style="text-align:left;">类别</td>                            2418
            <td>                                                              2419
              <select name="category">                                        2420
                {% for c in ["地方标准", "政策文件"] %}                       2421
                <option value="{{ c }}" {% if standard and                    2422
      standard.category == c %}selected{% endif %}>{{ c                       2423
      }}</option>                                                             2424
                {% endfor %}                                                  2425
              </select>                                                       2426
            </td></tr>                                                        2427
        <tr><td style="text-align:left;">发布时间</td>                        2428
            <td><input type="text" name="publish_date"                        2429
      value="{{ standard.publish_date or '' if standard else                  2430
      '' }}"></td></tr>                                                       2431
```

```
        <tr><td style="text-align:left;">实施时间</td>                    2432
            <td><input type="text" name="implement_date"                  2433
    value="{{ standard.implement_date or '' if standard                   2434
    else '' }}"></td></tr>                                                2435
        <tr><td style="text-align:left;vertical-align:top;">特点          2436
    /判定标准</td>                                                        2437
            <td><textarea name="summary"                                  2438
    style="width:420px;">{{ standard.summary or '' if                     2439
    standard else '' }}</textarea></td></tr>                              2440
    </table>                                                              2441
    <p style="margin-top:14px;">                                          2442
      <button type="submit" class="btn">保存</button>                     2443
      <a href="/standard/" class="btn gray">返回</a>                      2444
    </p>                                                                  2445
  </form>                                                                 2446
</div>                                                                    2447
{% endblock %}                                                            2448
#  ==== app/templates/standard/rules.html ====                            2449
{% extends "base.html" %}                                                 2450
{% block title %}结构化规则{% endblock %}                                 2451
{% block content %}                                                       2452
<div class="card">                                                        2453
  <h2>结构化规则 · {{ standard.province }} {{ standard.name               2454
    }}</h2>                                                               2455
  <p class="muted">                                                       2456
    将标准条文拆解为「判定项 + 比较符 + 阈值 + 条件类型」的可执行规则。                2457
  </p>                                                                    2458
  {% if rules %}                                                          2459
  <table>                                                                 2460
    <tr><th>判定项</th><th>比较符</th><th>阈值</th><th>条件类型</th><th>  2461
    适用分区</th><th>备注</th><th>操作</th></tr>                          2462
    {% for r in rules %}                                                  2463
    <tr>                                                                  2464
      <td>{{ r.item }}</td>                                               2465
      <td class="mono">{{ r.operator }}</td>                              2466
      <td class="mono">{{ r.threshold or "—" }}</td>                      2467
      <td>{{ r.condition_type }}</td>                                     2468
      <td>{{ r.region or "—" }}</td>                                      2469
      <td style="text-align:left;">{{ r.note or "—" }}</td>               2470
      <td>                                                                2471
        <form method="post" action="/standard/rule/{{ r.id                2472
    }}/delete" style="display:inline;">                                   2473
          {{ form.hidden_tag() }}                                         2474
          <button type="submit" class="btn small                          2475
    gray">删除</button>                                                   2476
        </form>                                                           2477
      </td>                                                               2478
    </tr>                                                                 2479
    {% endfor %}                                                          2480
  </table>                                                                2481
```

```
  {% else %}                                                                2482
  <p class="muted">暂无规则。</p>                                             2483
  {% endif %}                                                               2484
  <h3 style="font-size:14px;color:#1f3a5f;margin:20px 0                     2485
    8px;">添加规则</h3>                                                     2486
  <form method="post">                                                      2487
    {{ form.hidden_tag() }}                                                 2488
    <table style="width:auto;">                                             2489
      <tr>                                                                  2490
        <td style="text-align:left;">判定项</td>                            2491
        <td><input type="text" name="item"                                  2492
    placeholder="如：植被覆盖度" required></td>                             2493
        <td style="text-align:left;">比较符</td>                            2494
        <td>                                                                2495
          <select name="operator">                                          2496
            {% for o in [">=", ">", "<=", "<", "=="]                        2497
    %}<option value="{{ o }}">{{ o }}</option>{% endfor %}                  2498
          </select>                                                         2499
        </td>                                                               2500
      </tr>                                                                 2501
      <tr>                                                                  2502
        <td style="text-align:left;">阈值</td>                              2503
        <td><input type="text" name="threshold"                             2504
    placeholder="如：50"></td>                                              2505
        <td style="text-align:left;">条件类型</td>                          2506
        <td>                                                                2507
          <select name="condition_type">                                    2508
            <option value="必要条件">必要条件</option>                      2509
            <option value="充分条件">充分条件</option>                      2510
          </select>                                                         2511
        </td>                                                               2512
      </tr>                                                                 2513
      <tr>                                                                  2514
        <td style="text-align:left;">适用分区</td>                          2515
        <td><input type="text" name="region"                                2516
    placeholder="如宁夏分区"></td>                                          2517
        <td style="text-align:left;">备注</td>                              2518
        <td><input type="text" name="note"                                  2519
    style="width:260px;"></td>                                              2520
      </tr>                                                                 2521
    </table>                                                                2522
    <p style="margin-top:12px;">                                            2523
      <button type="submit" class="btn">添加规则</button>                   2524
      <a href="/standard/" class="btn gray">返回</a>                        2525
    </p>                                                                    2526
  </form>                                                                   2527
</div>                                                                      2528
{% endblock %}                                                              2529
# ==== app/templates/standard/compare.html ====                             2530
{% extends "base.html" %}                                                   2531
```

```
{% block title %}省际口径比对{% endblock %}
{% block content %}
<div class="card">
  <h2>省际口径比对</h2>
  <p class="muted">
    同一判定项在不同省份的阈值横向对比，避免人工翻阅多份标准文件。
  </p>
  {% if not groups %}
  <p class="muted">暂无规则数据。请先为标准录入规则，或运行 seed_demo.py 载入论文表 1 数据。</p>
  {% else %}
  {% for item, rs in groups.items() %}
  <h3 style="font-size:14px;color:#1f3a5f;margin:16px 0 8px;">{{ item }}</h3>
  <table>
    <tr><th>省份</th><th>标准/文件</th><th>条件</th><th>条件类型</th><th>备注</th></tr>
    {% for r in rs %}
    <tr>
      <td>{{ r.standard.province }}</td>
      <td style="text-align:left;">{{ r.standard.name }}</td>
      <td class="mono">{{ r.operator }} {{ r.threshold or "" }}</td>
      <td>{{ r.condition_type }}</td>
      <td style="text-align:left;">{{ r.note or "—" }}</td>
    </tr>
    {% endfor %}
  </table>
  {% endfor %}
  {% endif %}
  <p style="margin-top:12px;"><a href="/standard/" class="btn gray">返回</a></p>
</div>
{% endblock %}
# ==== app/templates/constraint/index.html ====
{% extends "base.html" %}
{% block title %}限制性判定{% endblock %}
{% block content %}
<div class="card">
  <h2>限制性指标判定</h2>
  <p class="muted">
    按论文表 2 与 4.1 节流程，对每项限制性条件逐项判定。
    任一项不通过即判定为「人工辅助或生态重建」，不再进入定量评价。
  </p>
  {% if not patches %}
  <p class="muted">暂无矿山图斑。</p>
  {% else %}
  <table>
    <tr><th>矿山图斑</th><th>判定结果</th><th>操作</th></tr>
```

```
      {% for p in patches %}                                                    2582
      <tr>                                                                      2583
        <td>{{ p.name }}</td>                                                   2584
        <td>                                                                    2585
          {% set r = latest.get(p.id) %}                                        2586
          {% if r %}                                                            2587
            {% if r.passed %}<span class="ok">通过</span>{%                     2588
      else %}<span class="warn">不通过</span>{% endif %}                        2589
            <span class="muted">({{ r.reason }})</span>                         2590
          {% else %}                                                            2591
            <span class="muted">未判定</span>                                   2592
          {% endif %}                                                           2593
        </td>                                                                   2594
        <td><a href="/constraint/{{ p.id }}">执行判定</a></td>                  2595
      </tr>                                                                     2596
      {% endfor %}                                                              2597
    </table>                                                                    2598
    {% endif %}                                                                 2599
</div>                                                                          2600
{% endblock %}                                                                  2601
# ==== app/templates/constraint/check.html ====                                 2602
{% extends "base.html" %}                                                       2603
{% block title %}限制性条件判定{% endblock %}                                   2604
{% block content %}                                                             2605
<div class="card">                                                              2606
  <h2>限制性条件判定 · {{ patch.name }}</h2>                                    2607
  <p class="muted">五项条件全部通过方可进入定量评价；任一项不通过即判为「人工辅助或生态重建」。</p 2608
    >                                                                           2609
  <form method="post">                                                          2610
    {{ form.hidden_tag() }}                                                     2611
    <table style="width:auto;">                                                 2612
      {% for field, name, val in field_states %}                                2613
      <tr>                                                                      2614
        <td style="text-align:left;">{{ name }}</td>                            2615
        <td style="text-align:left;">                                           2616
          <label><input type="radio" name="{{ field }}"                         2617
    value="1"                                                                   2618
            {% if val is none or val %}checked{% endif %}>                      2619
    通过</label>                                                                2620
                                                                      2621
          <label><input type="radio" name="{{ field }}"                         2622
    value="0"                                                                   2623
            {% if val is not none and not val %}checked{%                       2624
    endif %}> 不通过</label>                                                    2625
        </td>                                                                   2626
      </tr>                                                                     2627
      {% endfor %}                                                              2628
    </table>                                                                    2629
    <p style="margin-top:14px;">                                                2630
      <button type="submit" class="btn">保存判定</button>                       2631
```

```
        <a href="/constraint/" class="btn gray">返回</a>
      </p>
    </form>
    {% if record %}
    <div style="margin-top:16px;padding:12px;border-radius:4px
      ;
                  background:{{ '#eafaf1' if record.passed else
      '#fdecea' }};">
      <strong>判定结论：</strong>
      {% if record.passed %}
        <span class="ok">通过，可进入定量评价</span>
      {% else %}
        <span class="warn">不通过 —— 建议生态重建或人工辅助</span>
      {% endif %}
      <div class="muted" style="margin-top:6px;">{{
      record.reason }}</div>
    </div>
    {% endif %}
</div>
{% endblock %}
# ==== app/templates/ahp/index.html ====
{% extends "base.html" %}
{% block title %}AHP 权重计算{% endblock %}
{% block content %}
<div class="card">
    <h2>AHP 权重计算 · 选择指标层级</h2>
    <form method="post">
      {{ form.hidden_tag() }}
      <table style="width:auto;">
        <tr>
          <td style="text-align:left;">微地质单元类型</td>
          <td>{{ form.unit_type() }}</td>
        </tr>
        <tr>
          <td style="text-align:left;">指标层级</td>
          <td>{{ form.level() }}</td>
        </tr>
        <tr>
          <td style="text-align:left;">一级指标（层级为"二级指标"时选择）</td>
          <td>{{ form.l1() }}</td>
        </tr>
      </table>
      <p>{{ form.submit(class_="btn") }}</p>
    </form>
    <p class="muted">
      说明：选择单元类型与指标层级后，系统按论文表 3 确定判断矩阵规模。
      岩质边坡不设土壤指标，平台不设地形指标。
    </p>
</div>
{% endblock %}
```

```
# ==== app/templates/ahp/matrix.html ====
{% extends "base.html" %}
{% block title %}判断矩阵录入{% endblock %}
{% block content %}
<div class="card">
  <h2>
    判断矩阵录入
    · {{ unit_type }}
    · {% if level == 1 %}一级指标{% else %}二级指标（{{ l1 }}）{%
    endif %}
  </h2>
  <p class="muted">
    采用 1–9 标度：1 同等重要、3 稍微重要、5 明显重要、7 强烈重要、9 极端重要，
    2/4/6/8 为中间值。对角线为 1，下三角自动取倒数。下方已按论文表 3 权重预填。
  </p>
  <form method="post">
    {{ form.hidden_tag() }}
    <table>
      <tr>
        <th></th>
        {% for name in names %}<th>{{ name }}</th>{% endfor
    %}
      </tr>
      {% for i in range(names|length) %}
      <tr>
        <th>{{ names[i] }}</th>
        {% for j in range(names|length) %}
          {% if i == j %}
          <td class="muted">1</td>
          {% elif i < j %}
          <td><input type="number" step="0.01" min="0.01"
                     name="m_{{ i }}_{{ j }}" value="{{
    default[i][j] }}"></td>
          {% else %}
          <td class="muted">{{ default[i][j] }}</td>
          {% endif %}
        {% endfor %}
      </tr>
      {% endfor %}
    </table>
    <p>
      <button type="submit" class="btn">计算权重</button>
      <a href="/ahp/" class="btn" style="background:#8896a8;
    ">重新选择</a>
    </p>
  </form>
  {% if result %}
  <hr style="margin:20px 0;border:none;border-top:1px solid
    #e3e4e6;">
  <h2>计算结果</h2>
```

```
  <table style="width:auto;">                                          2732
    <tr><th>指标</th>{% for name in names %}<th>{{ name                2733
    }}</th>{% endfor %}</tr>                                           2734
    <tr>                                                               2735
      <th>权重</th>                                                    2736
      {% for w in result.weights %}<td class="mono">{{ w               2737
    }}</td>{% endfor %}                                                2738
    </tr>                                                              2739
  </table>                                                             2740
  <table style="width:auto;">                                          2741
    <tr>                                                               2742
      <th style="text-align:left;">最大特征值 λmax</th>                2743
      <td class="mono">{{ result.lambda_max }}</td>                    2744
    </tr>                                                              2745
    <tr>                                                               2746
      <th style="text-align:left;">一致性指标 CI</th>                  2747
      <td class="mono">{{ "%.4f"|format(result.ci) }}</td>             2748
    </tr>                                                              2749
    <tr>                                                               2750
      <th style="text-align:left;">一致性比例 CR</th>                  2751
      <td class="mono">{{ "%.4f"|format(result.cr) }}</td>             2752
    </tr>                                                              2753
    <tr>                                                               2754
      <th style="text-align:left;">一致性检验</th>                     2755
      <td>                                                             2756
        {% if result.consistent %}                                     2757
        <span class="ok">通过（CR &lt; 0.1）</span>                     2758
        {% else %}                                                     2759
        <span class="warn">不通过（CR ≥ 0.1）</span>                   2760
        {% endif %}                                                    2761
      </td>                                                            2762
    </tr>                                                              2763
    {% if not result.consistent and result.most_inconsistent2764
    %}                                                                 2765
    <tr>                                                               2766
      <th style="text-align:left;">建议优先调整</th>                   2767
      <td>                                                             2768
        第 {{ result.most_inconsistent.i + 1 }} 行、                   2769
        第 {{ result.most_inconsistent.j + 1 }} 列的比较               2770
        （当前 {{ result.most_inconsistent.value|round(3) }},          2771
        理想约 {{ result.most_inconsistent.expected|round(3)           2772
    }})                                                                2773
      </td>                                                            2774
    </tr>                                                              2775
    {% endif %}                                                        2776
  </table>                                                             2777
  {% endif %}                                                          2778
</div>                                                                 2779
{% endblock %}                                                         2780
# ==== app/templates/indicator/index.html ====                         2781
```

```
{% extends "base.html" %}
{% block title %}指标数据采集{% endblock %}
{% block content %}
<div class="card">
  <h2>指标数据采集与管理</h2>
  {% if not patches %}
  <p class="muted">暂无矿山图斑。请先运行 <span class="mono">python seed_demo.py</span> 载入案例，或由乙建立项目与图斑。</p>
  {% else %}
  {% for p in patches %}
  <div style="margin-bottom:20px;">
    <h3 style="font-size:15px;color:#1f3a5f;">{{ p.name }}（图斑：{{ p.name }}）</h3>
    <table>
      <tr><th>微地质单元</th><th>面积占比</th><th>数据状态</th><th>操作</th></tr>
      {% for u in p.units %}
      <tr>
        <td>{{ u.unit_type }}</td>
        <td>{% if u.ratio is not none %}{{ "%.2f%%"|format(u.ratio*100) }}{% else %}—{% endif %}</td>
        <td>
          {% set has_data = u.indicator_data %}
          {% if has_data %}<span class="ok">已录入</span>{% else %}<span class="muted">未录入</span>{% endif %}
        </td>
        <td><a href="/indicator/{{ u.id }}">录入/编辑</a></td>
      </tr>
      {% endfor %}
    </table>
  </div>
  {% endfor %}
  {% endif %}
</div>
{% endblock %}
# ==== app/templates/indicator/edit.html ====
{% extends "base.html" %}
{% block title %}指标数据录入{% endblock %}
{% block content %}
<div class="card">
  <h2>指标数据录入 · {{ unit.unit_type }}</h2>
  <p class="muted">
    说明：留空表示"不适用"（与"实测为 0"含义不同，计算时跳过）。
    岩质边坡无土壤指标、平台无地形指标，系统已按单元类型隐藏不相关字段。
  </p>
  <form method="post">
    {{ form.hidden_tag() }}
    {% for group_name, fields, has_hm in groups %}
    <h3 style="font-size:14px;color:#1f3a5f;margin:16px 0
```

```
8px;">{{ group_name }}</h3>
<table style="width:auto;">
  {% for fname, cname, unit_ in fields %}
  <tr>
    <td style="text-align:left;">{{ cname }}{% if unit_
%} ({{ unit_ }}) {% endif %}</td>
    {% if fname == "aspect" %}
    <td>
      <select name="aspect">
        <option value="">— 不适用 —</option>
        {% for a in aspect_types %}
        <option value="{{ a }}" {% if
current.get("aspect") == a %}selected{% endif %}>{{ a
}}</option>
        {% endfor %}
      </select>
    </td>
    {% else %}
    <td><input type="number" step="any" name="{{ fname
}}"
               value="{{ current.get(fname) if
current.get(fname) is not none else '' }}"></td>
    {% endif %}
  </tr>
  {% endfor %}
</table>
{% endfor %}
{% for group_name, fields, has_hm in groups %}
  {% if has_hm %}
  <h3 style="font-size:14px;color:#1f3a5f;margin:16px 0
8px;">重金属含量（mg/kg）</h3>
  <table style="width:auto;">
    <tr>
      {% for m in heavy_metals %}
      <th>{{ m }}</th>
      {% endfor %}
    </tr>
    <tr>
      {% for m in heavy_metals %}
      <td><input type="number" step="any" name="hm_{{ m
}}"
                 value="{{ current.get('heavy',
{}).get(m) if current.get('heavy', {}).get(m) is not
none else '' }}"></td>
      {% endfor %}
    </tr>
  </table>
  {% endif %}
{% endfor %}
<p style="margin-top:16px;">
```

```
        <button type="submit" class="btn">保存</button>
        <a href="/indicator/" class="btn"
      style="background:#8896a8;">返回</a>
      </p>
   </form>
</div>
{% endblock %}
# ==== app/templates/evaluate/index.html ====
{% extends "base.html" %}
{% block title %}适宜性评价{% endblock %}
{% block content %}
<div class="card">
   <h2>适宜性定量评价</h2>
   {% if not patches %}
   <p class="muted">暂无矿山图斑。请先运行 <span class="mono">python
      seed_demo.py</span> 载入案例。</p>
   {% else %}
   <table>
      <tr><th>矿山图斑</th><th>最近评价结果</th><th>操作</th></tr>
      {% for p in patches %}
      <tr>
         <td>{{ p.name }}</td>
         <td>
            {% if latest.get(p.id) %}
               F = {{ latest[p.id].final_score }}，{{
      latest[p.id].grade }}
            {% else %}
               <span class="muted">未评价</span>
            {% endif %}
         </td>
         <td>
            <a href="/evaluate/{{ p.id }}">执行评价</a>
            {% if latest.get(p.id) %}
             | <a href="/evaluate/result/{{
      latest[p.id].id }}">查看结果</a>
            {% endif %}
         </td>
      </tr>
      {% endfor %}
   </table>
   {% endif %}
</div>
{% endblock %}
# ==== app/templates/evaluate/run.html ====
{% extends "base.html" %}
{% block title %}执行评价{% endblock %}
{% block content %}
<div class="card">
   <h2>执行适宜性评价 · {{ patch.name }}</h2>
   <p class="muted">
```

```
    系统将按论文式（８）（９） 与表 3 权重，对图斑内各微地质单元做标准化、                    2932
    分单元赋分、面积加权合成，并判定适宜性等级。                                          2933
  </p>                                                                              2934
  <form method="post">                                                              2935
    {{ form.hidden_tag() }}                                                         2936
    <button type="submit" class="btn">开始评价</button>                              2937
    <a href="/evaluate/" class="btn" style="background:#8896 2938
    a8;">返回</a>                                                                   2939
  </form>                                                                           2940
</div>                                                                              2941
{% endblock %}                                                                      2942
# ==== app/templates/evaluate/result.html ====                                      2943
{% extends "base.html" %}                                                           2944
{% block title %}评价结果{% endblock %}                                               2945
{% block content %}                                                                 2946
<div class="card">                                                                  2947
  <h2>适宜性评价结果 · {{ patch.name if patch else "" }}</h2>                         2948
  <table style="width:auto;">                                                       2949
    <tr>                                                                            2950
      <th style="text-align:left;">最终评价因子 F</th>                                2951
      <td class="mono" style="font-size:18px;font-weight:700 2952
    ;">{{ r.final_score }}</td>                                                     2953
    </tr>                                                                           2954
    <tr>                                                                            2955
      <th style="text-align:left;">适宜性等级</th>                                    2956
      <td class="ok" style="font-size:18px;font-weight:700;" 2957
    >{{ r.grade }}</td>                                                             2958
    </tr>                                                                           2959
    <tr>                                                                            2960
      <th style="text-align:left;">推荐措施</th>                                      2961
      <td>{{ r.measures }}</td>                                                     2962
    </tr>                                                                           2963
  </table>                                                                          2964
  {% if unit_scores %}                                                              2965
  <h3 style="font-size:14px;color:#1f3a5f;margin:16px 0                             2966
    8px;">各微地质单元得分</h3>                                                       2967
  <table>                                                                           2968
    <tr><th>微地质单元</th><th>面积占比</th><th>单元评价因子                          2969
    F'</th><th>加权贡献</th></tr>                                                    2970
    {% for u in unit_scores %}                                                      2971
    <tr>                                                                            2972
      <td>{{ u.unit_type }}</td>                                                    2973
      <td>{{ "%.2f%%"|format(u.ratio*100) }}</td>                                   2974
      <td class="mono">{{ u.unit_score }}</td>                                      2975
      <td class="mono">{{ "%.4f"|format(u.ratio *                                   2976
    u.unit_score) }}</td>                                                           2977
    </tr>                                                                           2978
    {% endfor %}                                                                    2979
  </table>                                                                          2980
  {% endif %}                                                                       2981
```

```
  <p style="margin-top:16px;">                                                2982
    <a href="/evaluate/" class="btn" style="background:#8896 2983
    a8;">返回评价列表</a>                                                     2984
  </p>                                                                        2985
</div>                                                                        2986
{% endblock %}                                                                2987
# ==== app/templates/report/index.html ====                                   2988
{% extends "base.html" %}                                                     2989
{% block title %}报告输出{% endblock %}                                       2990
{% block content %}                                                           2991
<div class="card">                                                            2992
  <h2>报告与结果输出</h2>                                                     2993
  {% if not patches %}                                                        2994
  <p class="muted">暂无矿山图斑。请先运行 <span class="mono">python          2995
    seed_demo.py</span> 载入案例。</p>                                        2996
  {% else %}                                                                  2997
  <table>                                                                     2998
    <tr><th>矿山图斑</th><th>操作</th></tr>                                   2999
    {% for p in patches %}                                                    3000
    <tr>                                                                      3001
      <td>{{ p.name }}</td>                                                   3002
      <td>                                                                    3003
        <a href="/report/{{ p.id }}">查看得分明细</a>                         3004
         | <a href="/report/{{ p.id }}/word">导出                   3005
    Word 报告</a>                                                             3006
      </td>                                                                   3007
    </tr>                                                                     3008
    {% endfor %}                                                              3009
  </table>                                                                    3010
  {% endif %}                                                                 3011
</div>                                                                        3012
{% endblock %}                                                                3013
# ==== app/templates/report/detail.html ====                                  3014
{% extends "base.html" %}                                                     3015
{% block title %}得分明细{% endblock %}                                       3016
{% block content %}                                                           3017
<div class="card">                                                            3018
  <h2>得分明细 · {{ patch.name }}</h2>                                        3019
  <table style="width:auto;">                                                 3020
    <tr>                                                                      3021
      <th style="text-align:left;">最终评价因子 F</th>                        3022
      <td class="mono" style="font-size:18px;font-weight:700 3023
    ;">{{ "%.4f"|format(result.final_score) }}</td>                           3024
    </tr>                                                                     3025
    <tr>                                                                      3026
      <th style="text-align:left;">适宜性等级</th>                            3027
      <td class="ok" style="font-size:18px;font-weight:700;" 3028
    >{{ result.grade }}</td>                                                  3029
    </tr>                                                                     3030
    <tr>                                                                      3031
```

```
        <th style="text-align:left;">推荐措施</th>
        <td>{{ result.grade_info.measure }}</td>
      </tr>
    </table>
    <p style="margin-top:16px;">
      <a href="/report/{{ patch.id }}/word" class="btn">导出
      Word 报告</a>
      <a href="/report/" class="btn" style="background:#8896a8
      ;">返回</a>
    </p>
    {% for u in result.unit_results %}
    <h3 style="font-size:15px;color:#1f3a5f;margin:24px 0
      8px;">
      {{ u.unit_type }}（占比 {{ "%.2f%%"|format(u.ratio*100)
      }}）— 单元得分 {{ "%.4f"|format(u.unit_score) }}
    </h3>
    <table>
      <tr><th>一级指标</th><th>二级指标</th><th>实测值</th><th>标准化值</th><
      th>权重</th><th>贡献</th></tr>
      {% for d in u.details %}
      <tr>
        <td>{{ d.l1 }}</td>
        <td>{{ d.l2 }}</td>
        <td class="mono">{{ "—" if d.raw is none else d.raw
      }}</td>
        <td class="mono">{{ "—" if d.std_value is none else
      "%.4f"|format(d.std_value) }}</td>
        <td class="mono">{{ "%.4f"|format(d.weight) }}</td>
        <td class="mono">{{ "—" if d.contribution is none
      else "%.4f"|format(d.contribution) }}</td>
      </tr>
      {% endfor %}
    </table>
    <p class="muted" style="margin-top:6px;">
      一级指标得分：
      {% for k, v in u.l1_scores.items() %}{{ k }}={{
      "%.4f"|format(v) }} {% endfor %}
    </p>
    {% endfor %}
</div>
{% endblock %}
# ==== seed_demo.py ====
# -*- coding: utf-8 -*-
"""内置玉田县案例：把论文第 5 节案例数据写入数据库。
用于功能验证与操作演示。数据逐字取自论文表 5 与第 5 节。
运行：python seed_demo.py（幂等：已存在则先清空重建）
"""
import json
import sys
sys.stdout.reconfigure(encoding="utf-8")
```

```
from app import create_app
from app.models import (db, User, Project, MinePatch,
    ReferenceEcosystem,
                        MicroUnit, IndicatorData, Standard,
    StandardRule)
app = create_app()
# 论文表 1：11 项地方标准与政策文件
STANDARDS = [
    dict(category="地方标准", province="辽宁",
    name="矿山废弃地植被自然恢复技术规程",
        doc_number="DB21/T 3702—2023",
    publish_date="2023-03-30",
        implement_date="2023-04-30",
        summary="首个正式发布的矿山自然恢复地方标准；提出必要条件和充分条件",
        rules=[("植被覆盖度", ">=", "50", "必要条件", None),
               ("无地质灾害", "==", "是", "必要条件", None),
               ("无水土污染", "==", "是", "必要条件", None),
               ("与参照系差距", "<", "20", "充分条件", "%")]),
    dict(category="地方标准", province="湖北",
    name="废弃露天矿山生态环境自然恢复标准",
        doc_number="DB42/T 2421—2025",
    publish_date="2025-08-20",
        implement_date="2025-10-20",
        summary="结合矿山生态需求进行自然恢复适宜性分级判断",
        rules=[("自然恢复适宜性分级", "==", "是", "必要条件", None)]),
    dict(category="地方标准", province="内蒙古",
    name="历史遗留废弃矿山自然恢复判定规范",
        doc_number="DB15/T 4280—2026",
    publish_date="2026-01-30",
        implement_date="2026-03-01",
        summary="调查内容包含参照生态系统调查",
        rules=[("地质灾害发育程度", "<=", "弱发育", "必要条件", None),
               ("地形地貌与周边协调", "==", "是", "必要条件", None),
               ("有利于植被恢复的表层土壤", "==", "是", "必要条件", None),
               ("已形成稳定植被覆盖", "==", "是", "必要条件", None)]),
    dict(category="地方标准", province="江西",
    name="废弃矿山自然恢复技术规范",
        doc_number="DB36/T 2221—2026",
    publish_date="2026-02-02",
        implement_date="2026-08-01",
        summary="对参照生态系统进行了要求；提出自然恢复适宜性指数",
        rules=[("参照生态系统调查", "==", "是", "必要条件", None),
               ("自然恢复适宜性指数", ">=", "阈值", "充分条件", None)]),
    dict(category="地方标准", province="宁夏", name="矿山自然恢复认定规程",
        doc_number=None, publish_date="2026-06-18",
    implement_date=None,
        summary="分区差异化：三个一级生态修复分区及 22 个二级分区",
        rules=[("无地质灾害风险", "==", "是", "必要条件", None),
               ("植被覆盖度趋势", "==", "稳定或正增长", "必要条件", None),
               ("与参照系差距", "<", "20", "充分条件", "%")]),
```

```
    dict(category="政策文件", province="安徽",
        name="安徽省自然资源厅关于规范废弃露天矿山采取自然恢复方式进行治理的通知",
        doc_number="皖自然资修函[２０２０]３４号",
    publish_date="２０２０-０３-０３", implement_date=None,
        summary="自然恢复条件：责任主体为政府、无地质灾害、无水土污染、不在三线三边、水土条件良好",
        rules=[("责任主体", "==", "政府", "必要条件", None),
               ("无地质灾害", "==", "是", "必要条件", None),
               ("无水土污染", "==", "是", "必要条件", None)]),
    dict(category="政策文件", province="山东",
        name="山东省自然资源厅关于进一步规范历史遗留废弃露天开采矿山采取自然恢复方式进行治理的通知",
        doc_number="鲁自然资字[２０２２]１０４号",
    publish_date="２０２２-０８-０２", implement_date=None,
        summary="条件与安徽省相同，另增加自然恢复认定标准",
        rules=[("无地质灾害", "==", "是", "必要条件", None),
               ("无水土污染", "==", "是", "必要条件", None)]),
    dict(category="政策文件", province="湖南",
        name="关于印发《湖南省历史遗留矿山自然恢复指导意见（试行）》的通知",
        doc_number="湘自资办发[２０２３]５８号",
    publish_date="２０２３-０７-２０", implement_date=None,
        summary="自然恢复认定标准：无地质灾害或无威胁对象、无水土污染、设备拆除、符合规划、植被覆盖度
>７５%、连续３年正向增长",
        rules=[("植被覆盖度", ">", "７５", "必要条件", "%"),
               ("植被覆盖率趋势", "==", "连续３年正向增长", "必要条件", None),
               ("无地质灾害", "==", "是", "必要条件", None),
               ("符合国土空间规划", "==", "是", "必要条件", None)]),
    dict(category="政策文件", province="新疆",
        name="自治区历史遗留废弃矿山自然恢复方式销号图斑工作指南（试行）",
        doc_number="新自然资源办函〔２０２４〕２１９号",
    publish_date="２０２４-１２-２０", implement_date=None,
        summary="荒漠区沙地/裸土地/裸岩石砾地满足无地质灾害、地形协调、无水土污染即可判定",
        rules=[("植被覆盖度", ">=", "周边３０%", "必要条件", None),
               ("无地质灾害", "==", "是", "必要条件", None),
               ("地形平缓地貌稳定", "==", "是", "必要条件", None)]),
    dict(category="政策文件", province="广西",
        name="广西壮族自治区历史遗留矿山自然恢复技术指南",
        doc_number="桂自然资办〔２０２４〕２５０号",
    publish_date="２０２４-１２-３０", implement_date=None,
        summary="除高陡边坡和裸露岩体外，水土水热条件良好、植被自然恢复趋势明显",
        rules=[("无地质灾害", "==", "是", "必要条件", None),
               ("植被自然恢复趋势", "==", "明显", "必要条件", None),
               ("生产设备拆除完毕", "==", "是", "必要条件", None)]),
    dict(category="政策文件", province="西藏",
        name="西藏自治区历史遗留废弃矿山生态自然恢复认定指导意见（试行）",
        doc_number="藏自然资发〔２０２５〕１９号",
    publish_date="２０２５-０９-２４", implement_date=None,
        summary="含修复成本高、生态修复效益不显著等适用条件",
        rules=[("责任主体", "==", "政府", "必要条件", None),
               ("地形地貌完整与周边协调", "==", "是", "必要条件", None),
               ("无地质灾害", "==", "是", "必要条件", None)]),
]
```

```
CASE = {
    "name": "玉田县典型历史遗留废弃矿山案例",
    "patch_name": "玉田县孤树镇白云岩采石场",
    "area_ha": 5.6,
    "mineral_type": "白云岩",
    "units": [
        {
            "unit_type": "岩质边坡", "ratio": 0.1386,
            "data": {"coverage": 68.88, "shannon": 0.00,
                     "slope": 63.33, "aspect": "阴阳坡"},
        },
        {
            "unit_type": "碎石边坡", "ratio": 0.6295,
            "data": {"soil_ratio": 0.34, "thickness": 10,
    "ph": 9.24,
                     "organic_matter": 4.89,
                     "heavy_metals": {"Hg": 0.05, "As":
    5.17, "Ni": 12.60,
                                      "Cu": 11.70, "Cd":
    0.04, "Pb": 6.50},
                     "coverage": 87.92, "shannon": 0.60,
                     "slope": 28.75, "aspect": "阳坡"},
        },
        {
            "unit_type": "平台", "ratio": 0.2319,
            "data": {"soil_ratio": 0.56, "thickness": 30,
    "ph": 8.72,
                     "organic_matter": 10.92,
                     "heavy_metals": {"Hg": 0.02, "As":
    3.13, "Ni": 12.33,
                                      "Cu": 11.14, "Cd":
    0.09, "Pb": 9.98},
                     "coverage": 58.42, "shannon": 0.72},
        },
    ],
}
def clean_case():
    """清空案例相关数据（幂等重跑用）。
    indicator_data / score_details 等表未设级联删除，需显式清理。
    """
    from app.models import EvaluationResult, ScoreDetail,
    ConstraintCheck
    ScoreDetail.query.delete()
    EvaluationResult.query.delete()
    ConstraintCheck.query.delete()
    IndicatorData.query.delete()
    MicroUnit.query.delete()
    ReferenceEcosystem.query.delete()
    MinePatch.query.delete()
    Project.query.delete()
```

```
    db.session.commit()
def seed():
    with app.app_context():
        clean_case()
        admin = User.query.filter_by(username="admin").first()
        if not admin:
            from werkzeug.security import generate_password_hash
            admin = User(username="admin",
                         password_hash=generate_password_hash("admin123"),
                         role="admin")
            db.session.add(admin)
            db.session.commit()
        project = Project(name=CASE["name"], code="DEMO-001",
                          description="论文第 5 节典型案例", created_by=admin.id)
        db.session.add(project)
        db.session.flush()
        patch = MinePatch(project_id=project.id, name=CASE["patch_name"],
                          admin_region="河北省唐山市玉田县",
                          area_ha=CASE["area_ha"],
                          mineral_type=CASE["mineral_type"],
                          mining_method="露天开采", stop_years=None,
                          responsible_entity_missing=True)
        db.session.add(patch)
        db.session.flush()
        ref = ReferenceEcosystem(patch_id=patch.id,
                                 coverage_benchmark=75.0, shannon_benchmark=0.7)
        db.session.add(ref)
        for u in CASE["units"]:
            unit = MicroUnit(patch_id=patch.id, unit_type=u["unit_type"],
                             ratio=u["ratio"], area=None)
            db.session.add(unit)
            db.session.flush()
            d = u["data"]
            ind = IndicatorData(
                unit_id=unit.id,
                soil_ratio=d.get("soil_ratio"),
                thickness=d.get("thickness"),
                ph=d.get("ph"),
                organic_matter=d.get("organic_matter"),
                heavy_metals=json.dumps(d["heavy_metals"], ensure_ascii=False)
```

```
                                    if d.get("heavy_metals") else
    None,
                    coverage=d.get("coverage"),
                    shannon=d.get("shannon"),
                    slope=d.get("slope"),
                    aspect=d.get("aspect"),
                )
                db.session.add(ind)
            db.session.commit()
            print("已写入案例：", CASE["name"])
            print("  图斑：", CASE["patch_name"])
            print("  微地质单元：岩质边坡 13.86% / 碎石边坡 62.95% / 平台
    23.19%")
            print("  管理员账号：admin / admin123")
            seed_standards()
def seed_standards():
    """写入论文表 1 的 11 项标准与政策文件及其结构化规则。"""
    StandardRule.query.delete()
    Standard.query.delete()
    db.session.commit()
    for s in STANDARDS:
        std = Standard(category=s["category"],
    province=s["province"],
                       name=s["name"],
    doc_number=s["doc_number"],
                       publish_date=s["publish_date"],
                       implement_date=s["implement_date"],
                       summary=s["summary"])
        db.session.add(std)
        db.session.flush()
        for item, op, thr, ctype, note in s["rules"]:
            db.session.add(StandardRule(standard_id=std.id,
    item=item,
                                        operator=op,
    threshold=thr,

    condition_type=ctype, note=note))
    db.session.commit()
    print("已写入标准/政策文件：%d 项，规则 %d 条"
          % (len(STANDARDS), sum(len(s["rules"]) for s in
    STANDARDS)))
if __name__ == "__main__":
    seed()
# ==== test_ahp.py ====
# -*- coding: utf-8 -*-
"""地基自检：验证 AHP 引擎与应用可启动。"""
import sys
sys.stdout.reconfigure(encoding="utf-8")
import numpy as np
from app.services import ahp_service as ahp
```

```
LINE = "=" * 66
# ----------------------------------------------------------------
    ----------
# 1） 完全一致矩阵 → 权重应精确还原，CR=0
# ----------------------------------------------------------------
    ----------
print(LINE)
print("测试 1  完全一致判断矩阵的权重还原与一致性")
print(LINE)
# 由权重构造完全一致矩阵：a_ij = w_i / w_j
w_true = np.array([0.44, 0.31, 0.25])
A = np.outer(w_true, 1.0 / w_true)
r = ahp.weights_and_consistency(A)
print("  判断矩阵 A：")
for row in A:
    print("    " + "  ".join(f"{v:8.4f}" for v in row))
print()
print("  计算权重 :", "  ".join(f"{v:.4f}" for v in
    r["weights"]))
print("  真实权重 :", "  ".join(f"{v:.4f}" for v in w_true))
print("  λmax = %.6f   CI = %.3e   CR = %.3e   一致: %s"
      % (r["lambda_max"], r["ci"], r["cr"],
    r["consistent"]))
assert np.allclose(r["weights"], w_true, atol=1e-6),
    "权重未精确还原"
assert r["consistent"], "完全一致矩阵应通过检验"
# ----------------------------------------------------------------
    ----------
# 2） 上三角输入 → 完整矩阵 → 一致性定位
# ----------------------------------------------------------------
    ----------
print()
print(LINE)
print("测试 2  上三角录入与最不一致项定位")
print(LINE)
# 一个不太一致的 3 阶矩阵：上三角 [2, 5, 3]（a12=2, a13=5, a23=3）
M = ahp.matrix_from_upper([2, 5, 3])
r2 = ahp.weights_and_consistency(M)
bad = ahp.find_most_inconsistent(M)
print("  由 [a12=2, a13=5, a23=3] 构造矩阵：")
for row in M:
    print("    " + "  ".join(f"{v:8.4f}" for v in row))
print("  权重:", "  ".join(f"{v:.4f}" for v in r2["weights"]),
      "  CR = %.3f  一致: %s" % (r2["cr"], r2["consistent"]))
print("  最不一致比较项: 第(%d,%d) 项，当前 %.3f，理想 %.3f"
      % (bad["i"] + 1, bad["j"] + 1, bad["value"],
    bad["expected"]))
# ----------------------------------------------------------------
    ----------
# 3） 多专家聚合（几何平均）
```

```
# ----------------------------------------------------------------------3382
    ----------------                                                    3383
print()                                                                 3384
print(LINE)                                                             3385
print("测试 3  多专家判断矩阵聚合（几何平均）")                          3386
print(LINE)                                                             3387
m1 = ahp.matrix_from_upper([2, 4, 3])                                   3388
m2 = ahp.matrix_from_upper([3, 5, 2])                                   3389
agg = ahp.aggregate_matrices([m1, m2])                                  3390
print("  专家1 上三角 [2,4,3]，专家2 上三角 [3,5,2]")                   3391
print("  聚合后上三角： a12=%.4f a13=%.4f a23=%.4f"                     3392
      % (agg[0, 1], agg[0, 2], agg[1, 2]))                              3393
assert abs(agg[0, 1] - np.sqrt(2 * 3)) < 1e-9, "几何平均错误"           3394
# ----------------------------------------------------------------------3395
    ----------------                                                    3396
# 4） 论文表 3 权重可还原性                                             3397
# ----------------------------------------------------------------------3398
    ----------------                                                    3399
print()                                                                 3400
print(LINE)                                                             3401
print("测试 4  论文表 3 碎石边坡一级权重 (0.44, 0.31, 0.25)")           3402
print(LINE)                                                             3403
j = ahp.judge_from_weights([0.44, 0.31, 0.25])                          3404
print("  λmax = %.6f  CR = %.3e  一致： %s" %                           3405
    (j["lambda_max"], j["cr"], j["consistent"]))                        3406
assert j["consistent"], "论文权重应能作为完全一致矩阵主特征向量"       3407
# ----------------------------------------------------------------------3408
    ----------------                                                    3409
# 5） 应用可启动 + 建表                                                 3410
# ----------------------------------------------------------------------3411
    ----------------                                                    3412
print()                                                                 3413
print(LINE)                                                             3414
print("测试 5  Flask 应用启动与建表")                                   3415
print(LINE)                                                             3416
from app import create_app                                              3417
from app.models import db                                               3418
app = create_app()                                                      3419
with app.app_context():                                                 3420
    db.create_all()                                                     3421
    tables = sorted(db.metadata.tables.keys())                          3422
print("  已创建 %d 张表：" % len(tables))                               3423
for t in tables:                                                        3424
    print("    -", t)                                                  3425
print()                                                                 3426
print(LINE)                                                             3427
print("全部测试通过 √")                                                3428
print(LINE)                                                             3429
# ==== test_evaluate.py ====                                           3430
# -*- coding: utf-8 -*-                                                3431
```

```
"""评价引擎自检：用论文玉田县案例跑通完整计算链。"""
import sys
sys.stdout.reconfigure(encoding="utf-8")
from app.services import evaluation_service as ev
from app.services import standardization as std
LINE = "=" * 70
# ------------------------------------------------------------------
# 论文第 5 节案例数据（表 5，逐字抄录）
# ------------------------------------------------------------------
# 三类单元面积占比：岩质边坡 １３．８６%、碎石边坡 ６２．９５%、平台 ２３．１９%
# 注：岩质边坡无土壤指标、平台无地形指标，对应"不适用"。
case = [
    {
        "unit_type": "岩质边坡",
        "ratio": 0.1386,
        "data": {
            "coverage": 68.88, "shannon": 0.00,
            "slope": 63.33, "aspect": "阴阳坡",
        },
    },
    {
        "unit_type": "碎石边坡",
        "ratio": 0.6295,
        "data": {
            "soil_ratio": 0.34, "thickness": 10, "ph":
    9.24, "organic_matter": 4.89,
            "heavy_metals": {"Hg": 0.05, "As": 5.17, "Ni":
    12.60,
                             "Cu": 11.70, "Cd": 0.04, "Pb":
    6.50},
            "coverage": 87.92, "shannon": 0.60,
            "slope": 28.75, "aspect": "阳坡",
        },
    },
    {
        "unit_type": "平台",
        "ratio": 0.2319,
        "data": {
            "soil_ratio": 0.56, "thickness": 30, "ph":
    8.72, "organic_matter": 10.92,
            "heavy_metals": {"Hg": 0.02, "As": 3.13, "Ni":
    12.33,
                             "Cu": 11.14, "Cd": 0.09, "Pb":
    9.98},
            "coverage": 58.42, "shannon": 0.72,
        },
    },
]
```

```
# ------------------------------------------------------------------------
    ----------------
# 逐单元输出标准化与得分
# ------------------------------------------------------------------------
    ----------------
print(LINE)
print("玉田县案例 · 评价引擎完整计算")
print(LINE)
for u in case:
    r = ev.score_unit(u["unit_type"], u["data"])
    print()
    print("  【%s】 (面积占比 %.2f%%)" % (u["unit_type"],
    u["ratio"] * 100))
    print("  " + "-" * 60)
    for l1, score in r["l1_scores"].items():
        print("    %s 得分 = %.4f" % (l1, score))
    print("    单元评价因子 F' = %.4f" % r["unit_score"])
print()
print(LINE)
print("面积加权合成（式(9)）")
print(LINE)
res = ev.score_mine(case)
total = res["final_score"]
print()
for ur in res["unit_results"]:
    print("  %s %.2f%% × %.4f = %.4f"
          % (ur["unit_type"], ur["ratio"] * 100,
    ur["unit_score"], ur["weighted"]))
print("  " + "-" * 60)
print("  最终评价因子 F = %.4f" % total)
print()
print("  适宜性等级：%s" % res["grade"])
print("  判定依据 ：%s" % res["grade_info"]["meaning"])
print("  推荐措施 ：%s" % res["grade_info"]["measure"])
print()
print(LINE)
print("结论")
print(LINE)
print("  - 评价引擎按论文式(8)(9) 与表 3 权重完整跑通")
print("  - 坡度标准化采用论文 25° 分界的线性折减默认值（见 standardization.py）")
print("  - 最终 F ≈ %.2f，判定为「%s」，与论文「最终适宜自然恢复」的结论一致"
      % (total, res["grade"]))
print("  - 论文表 5 所载最终得分为 0.76；差异集中在坡度、坡向的标准化口径，")
print("    该口径在 standardization.std_slope / aspect_score
    中集中定义、可配置")
print(LINE)
# ==== test_web_ahp.py ====
# -*- coding: utf-8 -*-
"""M6 AHP 网页界面自检：走通选择→录入→计算 全流程。"""
import sys
```

```
sys.stdout.reconfigure(encoding="utf-8")
from app import create_app
LINE = "=" * 66
# 测试配置：关闭 CSRF 以简化测试客户端（真实运行仍开启 CSRF）
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///:memory:"
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
client = app.test_client()
# 建立表并创建测试用户，登录后再访问业务页面
with app.app_context():
    from app.models import db, User
    from werkzeug.security import generate_password_hash
    db.create_all()
    if not User.query.filter_by(username="admin").first():
        db.session.add(User(username="admin",
                            password_hash=generate_password_
    hash("admin123"),
                            role="admin"))
        db.session.commit()
client.post("/login", data={"username": "admin",
    "password": "admin123"})
print(LINE)
print("测试 1  首页与 AHP 入口")
print(LINE)
r = client.get("/")
print("  首页状态码：", r.status_code, "（应 200）")
assert r.status_code == 200
r = client.get("/ahp/")
print("  AHP 入口状态码：", r.status_code, "（应 200）")
assert r.status_code == 200
assert "微地质单元类型" in r.get_data(as_text=True)
print()
print(LINE)
print("测试 2  选择碎石边坡一级指标 → 跳转矩阵页")
print(LINE)
r = client.post("/ahp/", data={
    "unit_type": "碎石边坡", "level": "1", "submit": "生成判断矩阵",
}, follow_redirects=True)
body = r.get_data(as_text=True)
print("  跳转后页面含「判断矩阵录入」：", "判断矩阵录入" in body)
print("  含三个一级指标（土壤 / 植被 / 地形）：", all(n in body for n in
    ("土壤指标", "植被指标", "地形指标")))
assert "判断矩阵录入" in body
assert all(n in body for n in ("土壤指标", "植被指标", "地形指标"))
print()
print(LINE)
```

```
print("测试 3  提交论文表 3 一致矩阵 → 还原权重")
print(LINE)
# 碎石边坡一级权重 （0.44, 0.31, 0.25） 对应的完全一致矩阵上三角：
# a12 = 0.44/0.31 = 1.4194, a13 = 0.44/0.25 = 1.76, a23 =
    0.31/0.25 = 1.24
r = client.post("/ahp/matrix?unit_type=碎石边坡&level=1", data={
    "m_0_1": "1.4194", "m_0_2": "1.76", "m_1_2": "1.24",
}, follow_redirects=True)
body = r.get_data(as_text=True)
print("  结果页含「计算结果」:", "计算结果" in body)
print("  含权重 0.44:", "0.44" in body, " 含 0.31:", "0.31" in
    body, " 含 0.25:", "0.25" in body)
print("  含一致性检验通过:", "通过" in body)
assert "计算结果" in body
assert "0.44" in body and "0.31" in body and "0.25" in body
print()
print(LINE)
print("测试 4  不一致矩阵应提示不通过并定位")
print(LINE)
# 明显不一致：a12=9, a13=1, a23=9
r = client.post("/ahp/matrix?unit_type=碎石边坡&level=1", data={
    "m_0_1": "9", "m_0_2": "1", "m_1_2": "9",
}, follow_redirects=True)
body = r.get_data(as_text=True)
print("  含「不通过」:", "不通过" in body)
print("  含「建议优先调整」:", "建议优先调整" in body)
assert "不通过" in body
assert "建议优先调整" in body
print()
print(LINE)
print("M6 网页界面全部测试通过 √")
print(LINE)
# ==== test_web_modules.py ====
# -*- coding: utf-8 -*-
"""M1–M5 模块自检：登录、项目档案、微地质单元、标准库、限制性判定。"""
import os
import sys
sys.stdout.reconfigure(encoding="utf-8")
from app import create_app
LINE = "=" * 66
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" + os.path.join(
        os.path.dirname(__file__), "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
client = app.test_client()
print(LINE)
```

```
print("测试 1  未登录应被重定向到登录页")
print(LINE)
r = client.get("/project/")
print("  /project/ 状态码:", r.status_code, " 重定向到:",
      r.headers.get("Location"))
assert r.status_code == 302 and "login" in
      r.headers.get("Location", "")
print()
print(LINE)
print("测试 2  登录")
print(LINE)
r = client.post("/login", data={"username": "admin",
      "password": "admin123"},
                  follow_redirects=True)
body = r.get_data(as_text=True)
print("  登录后含「系统功能」:", "系统功能" in body)
print("  含用户名 admin:", "admin" in body)
assert "系统功能" in body
print()
print(LINE)
print("测试 3  M2 项目与矿山档案")
print(LINE)
r = client.get("/project/")
body = r.get_data(as_text=True)
print("  项目列表状态码:", r.status_code)
print("  含案例项目:", "玉田县典型历史遗留废弃矿山案例" in body)
assert "玉田县典型历史遗留废弃矿山案例" in body
# 新建项目
client.post("/project/add", data={"name": "测试项目A", "code":
      "T-001"})
r = client.get("/project/")
print("  新建项目后含「测试项目A」:", "测试项目A" in r.get_data(as_text=True
      ))
assert "测试项目A" in r.get_data(as_text=True)
print()
print(LINE)
print("测试 4  M5 微地质单元（面积占比自动计算）")
print(LINE)
from app.models import MinePatch
with app.app_context():
      patch = MinePatch.query.filter_by(name="玉田县孤树镇白云岩采石场").f
      irst()
      pid = patch.id
r = client.get("/unit/%d" % pid)
body = r.get_data(as_text=True)
print("  单元列表含三类单元:", all(t in body for t in ("岩质边坡",
      "碎石边坡", "平台")))
print("  含占比合计:", "占比合计" in body)
assert "岩质边坡" in body and "占比合计" in body
print()
```

```
print(LINE)
print("测试 5  M3 标准知识库与省际口径比对")
print(LINE)
r = client.get("/standard/")
body = r.get_data(as_text=True)
print("  含辽宁/湖南/新疆标准:", all(p in body for p in ("辽宁", "湖南",
    "新疆")))
assert "辽宁" in body
r = client.get("/standard/compare")
body = r.get_data(as_text=True)
print("  口径比对页含「植被覆盖度」分组:", "植被覆盖度" in body)
print("  含多省份并列:", "湖南" in body and "新疆" in body)
assert "植被覆盖度" in body
print()
print(LINE)
print("测试 6  M4 限制性指标判定（一票否决）")
print(LINE)
r = client.get("/constraint/%d" % pid)
body = r.get_data(as_text=True)
print("  含五项条件:", all(x in body for x in ("政策约束", "区位约束",
    "土壤污染", "地质灾害", "耕地属性")))
assert all(x in body for x in ("政策约束", "区位约束", "土壤污染",
    "地质灾害", "耕地属性"))
# 全部通过
client.post("/constraint/%d" % pid, data={
    "policy_pass": "1", "location_pass": "1",
    "pollution_pass": "1",
    "geo_pass": "1", "farmland_pass": "1"})
r = client.get("/constraint/%d" % pid)
body = r.get_data(as_text=True)
print("  全通过后显示「通过，可进入定量评价」:", "可进入定量评价" in body)
assert "可进入定量评价" in body
# 一项不通过
client.post("/constraint/%d" % pid, data={
    "policy_pass": "1", "location_pass": "0",
    "pollution_pass": "1",
    "geo_pass": "1", "farmland_pass": "1"})
r = client.get("/constraint/%d" % pid)
body = r.get_data(as_text=True)
print("  一项不通过后显示「不通过」:", "不通过" in body)
print("  含失败项「区位约束」:", "区位约束" in body)
assert "不通过" in body
print()
print(LINE)
print("M1–M5 全部测试通过 √")
print(LINE)
# ==== test_web_evaluate.py ====
# -*- coding: utf-8 -*-
"""M7 + M8 端到端自检：录数据 → 执行评价 → 查看结果。"""
import os
```

```
import sys
sys.stdout.reconfigure(encoding="utf-8")
from app import create_app
LINE = "=" * 66
# 与真实配置共用同一个 app.db，但关闭 CSRF 便于测试客户端
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" + os.path.join(
        os.path.dirname(__file__), "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
client = app.test_client()
# 登录（案例种子已创建 admin 用户）
client.post("/login", data={"username": "admin",
    "password": "admin123"})
print(LINE)
print("测试 1  M7 指标数据列表")
print(LINE)
r = client.get("/indicator/")
body = r.get_data(as_text=True)
print("  状态码：", r.status_code)
print("  含案例图斑：", "玉田县孤树镇白云岩采石场" in body)
print("  含三个单元：", all(n in body for n in ("岩质边坡", "碎石边坡",
    "平台")))
assert r.status_code == 200
assert "岩质边坡" in body and "碎石边坡" in body and "平台" in body
print()
print(LINE)
print("测试 2  M7 指标数据录入页（按单元类型显示字段）")
print(LINE)
from app.models import MicroUnit, MinePatch
with app.app_context():
    unit_rock = MicroUnit.query.filter_by(unit_type="岩质边坡").
    first()
    unit_gravel = MicroUnit.query.filter_by(unit_type="碎石边坡"
    ).first()
r = client.get("/indicator/%d" % unit_rock.id)
b_rock = r.get_data(as_text=True)
print("  岩质边坡页：无「土石比」字段：", "土石比" not in b_rock, "（应 True）")
print("  岩质边坡页：含「植被覆盖度」：", "植被覆盖度" in b_rock)
assert "土石比" not in b_rock and "植被覆盖度" in b_rock
r = client.get("/indicator/%d" % unit_gravel.id)
b_gravel = r.get_data(as_text=True)
print("  碎石边坡页：含「土石比」字段：", "土石比" in b_gravel)
print("  碎石边坡页：含 8 种重金属：", all(m in b_gravel for m in
    ("Cd", "Hg", "Pb", "Cr", "As", "Ni", "Cu", "Zn")))
assert "土石比" in b_gravel and all(m in b_gravel for m in
    ("Cd", "Hg", "Pb", "Cr", "As", "Ni", "Cu", "Zn"))
```

```
print()
print(LINE)
print("测试 3  M8 执行评价")
print(LINE)
with app.app_context():
    patch = MinePatch.query.first()
r = client.post("/evaluate/%d" % patch.id, data={},
    follow_redirects=True)
body = r.get_data(as_text=True)
print("  跳转至结果页：", "适宜性评价结果" in body)
print("  含最终评价因子 F：", "最终评价因子 F" in body)
print("  含等级「适宜」：", "适宜" in body)
print("  含推荐措施「自然恢复」：", "自然恢复" in body)
assert "最终评价因子 F" in body and "自然恢复" in body
print()
print(LINE)
print("测试 4  结果持久化")
print(LINE)
from app.models import EvaluationResult
with app.app_context():
    res = EvaluationResult.query.order_by(EvaluationResult.i
    d.desc()).first()
    print("  最终得分 F =", res.final_score)
    print("  等级 =", res.grade)
    assert res.grade == "适宜"
    assert 0.7 <= res.final_score < 1.0
print()
print(LINE)
print("M7 + M8 端到端全部测试通过 √")
print(LINE)
# ==== test_web_report.py ====
# -*- coding: utf-8 -*-
"""M9 报告输出自检：得分明细页 + Word 导出。"""
import os
import sys
import io
sys.stdout.reconfigure(encoding="utf-8")
from app import create_app
LINE = "=" * 66
class TestConfig:
    SECRET_KEY = "test"
    SQLALCHEMY_DATABASE_URI = "sqlite:///" + os.path.join(
        os.path.dirname(__file__), "app", "app.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    WTF_CSRF_ENABLED = False
    TESTING = True
app = create_app(TestConfig)
client = app.test_client()
# 登录（案例种子已创建 admin 用户）
client.post("/login", data={"username": "admin",
```

```
        "password": "admin123"})
from app.models import MinePatch
with app.app_context():
        patch = MinePatch.query.first()
        pid = patch.id
print(LINE)
print("测试 1  报告列表")
print(LINE)
r = client.get("/report/")
print("  状态码:", r.status_code)
assert r.status_code == 200
print()
print(LINE)
print("测试 2  得分明细页")
print(LINE)
r = client.get("/report/%d" % pid)
body = r.get_data(as_text=True)
print("  状态码:", r.status_code)
print("  含「得分明细」:", "得分明细" in body)
print("  含「最终评价因子 F」:", "最终评价因子 F" in body)
print("  含二级指标明细（植被覆盖度/坡度/坡向）:", all(n in body for n in
        ("植被覆盖度", "坡度", "坡向")))
print("  含权重/标准化值列:", "标准化值" in body and "贡献" in body)
assert r.status_code == 200
assert "植被覆盖度" in body and "标准化值" in body
print()
print(LINE)
print("测试 3  Word 报告导出")
print(LINE)
r = client.get("/report/%d/word" % pid)
print("  状态码:", r.status_code)
print("  Content-Type:", r.headers.get("Content-Type",
        "")[:60])
data = r.data
print("  文件大小:", len(data), "字节")
print("  是否为有效 docx（PK 头）:", data[:2] == b"PK")
assert r.status_code == 200 and data[:2] == b"PK"
# 用 python-docx 打开验证内容
import docx
d = docx.Document(io.BytesIO(data))
text = "\n".join(p.text for p in d.paragraphs)
print("  含报告标题:", "自然恢复适宜性评价报告" in text)
print("  含评价结论章节:", "评价结论" in text)
print("  含得分明细章节:", "得分明细" in text)
print("  段落数:", len(d.paragraphs), " 表格数:", len(d.tables))
assert "自然恢复适宜性评价报告" in text and len(d.tables) >= 3
print()
print(LINE)
print("M9 报告输出全部测试通过 √")
print(LINE)
```